



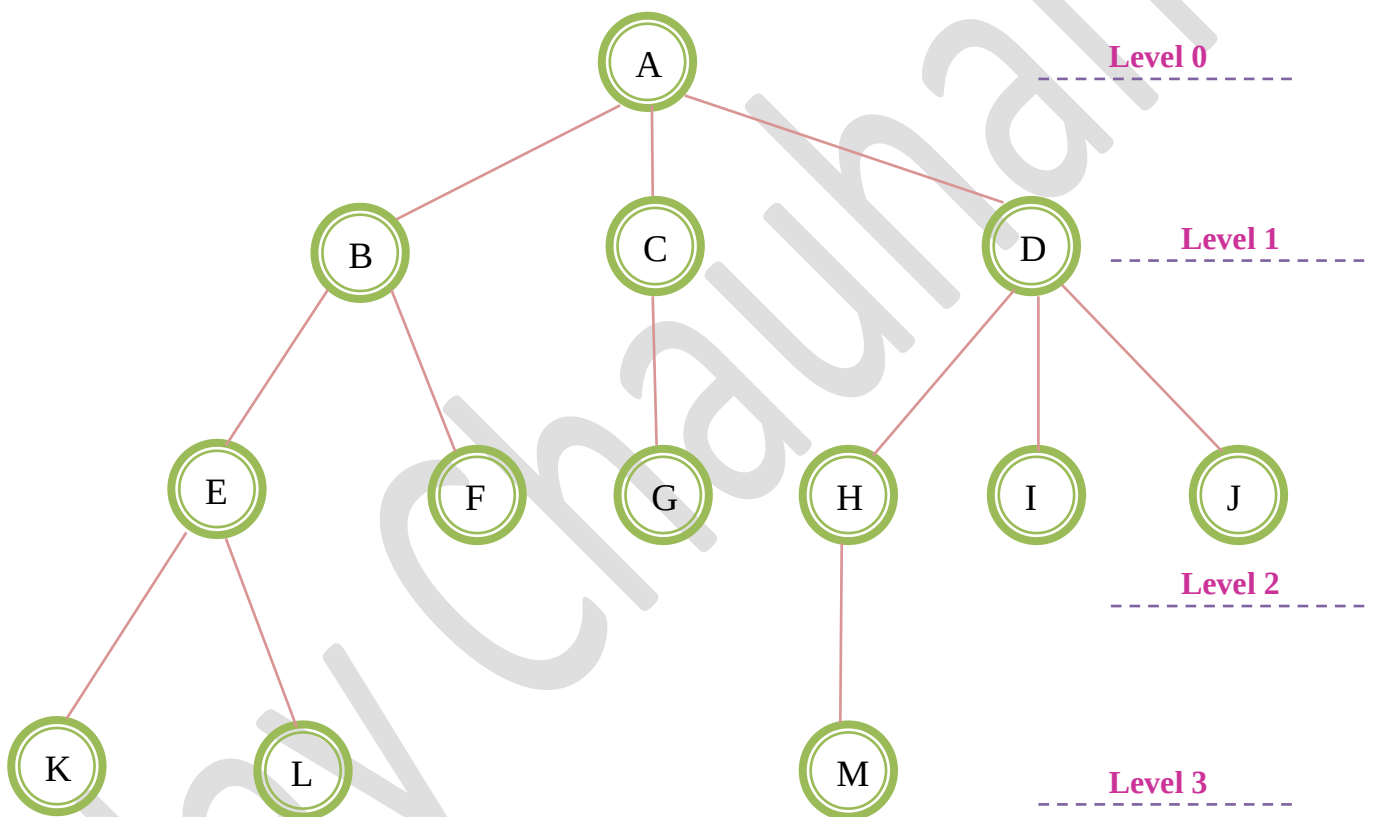
Semester: 2

Subject : Data Structures using Java

Chapter : Tree

❖ Introduction to Tree Data Structure

- Tree Data Structure is a non-linear data structure in which a collection of elements known as nodes are connected to each other via edges such that there exists exactly one path between any two nodes.
- A tree is a nonlinear hierarchical data structure that consists of nodes connected by edges.
- Different tree data structures allow quicker and easier access to the data as it is a non-linear data structure.



❖ Properties of Trees

➤ Root Node

- The topmost node of the tree is called the **Root Node**.
- Root Node = A

➤ Parent Node

- A node having one or more child nodes is called a Parent Node.
- Parent Nodes are:
 - A → Parent of B, C, D
 - B → Parent of E, F



Semester: 2

Subject : Data Structures using Java

Chapter : Tree

- C → Parent of G
- D → Parent of H, I, J
- E → Parent of K, L
- H → Parent of M

➤ **Child Node**

→ Nodes directly below a parent node are called Child Nodes.

→ Examples:

- B, C, D are children of A
- E and F are children of B
- G is child of C
- H, I, J are children of D
- K and L are children of E
- M is child of H

➤ **Leaf Node**

→ Nodes having no children are called Leaf Nodes.

→ Leaf Nodes are:

- F
- G
- I
- J
- K
- L
- M

➤ **Ancestor of a Node**

→ All nodes from root to a particular node are called its Ancestors.

→ Examples:

- Ancestors of G = A, C
- Ancestors of M = A, D, H
- Ancestors of K = A, B, E
- Ancestors of L = A, B, E

➤ **Descendant of a Node**

→ All nodes below a particular node are called its Descendants.

→ Examples:

- Descendants of A = B, C, D, E, F, G, H, I, J, K, L, M
- Descendants of B = E, F, K, L
- Descendants of D = H, I, J, M
- Descendants of E = K, L



Semester: 2

Subject : Data Structures using Java

Chapter : Tree

➤ Sibling Nodes

→ Nodes having the same parent are called Siblings.

→ Sibling Groups:

- B, C, D
- E and F
- H, I, J
- K and L

➤ Level of Node

→ Level = Number of nodes from root to that node.

Node	Level
A	0
B, C, D	1
E, F, G, H, I, J	2
K, L, M	3

➤ Internal Node

→ Nodes having at least one child are called Internal Nodes.

→ Internal Nodes are:

- A
- B
- C
- D
- E
- H

➤ Height of a Node

→ Height = Longest path from that node to a leaf node.

Node	Height
K, L, M	0
F, G, I, J	0
E	1
H	1
B	2
C	1
D	2
A	3



Semester: 2

Subject : Data Structures using Java

Chapter : Tree

➤ Height of Tree

→ Height of tree = Height of root node.

- Height of Tree = 3

➤ Depth of a Node

→ Depth = Number of edges from root to that node.

Node	Depth
A	0
B, C, D	1
E, F, G, H, I, J	2
K, L, M	3

➤ Depth of Tree

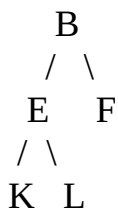
→ Maximum depth among all nodes.

- Depth of Tree = 3

➤ Subtree

→ A node with all its descendants forms a Subtree.

Subtree of B



Subtree of D



➤ Neighbor of a Node

→ Nodes directly connected to a node are called its Neighbors.

→ Examples:

- Neighbors of A = B, C, D
- Neighbors of D = A, H, I, J
- Neighbors of E = B, K, L
- Neighbors of M = H



Semester: 2

Subject : Data Structures using Java

Chapter : Tree

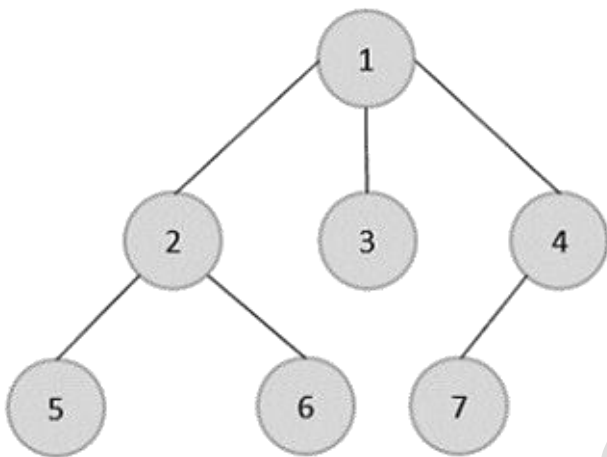
- Neighbors of G = C

❖ Types of Trees

1. General Tree

→ A tree in which a node can have **any number of children** is called a **General Tree**.

Example



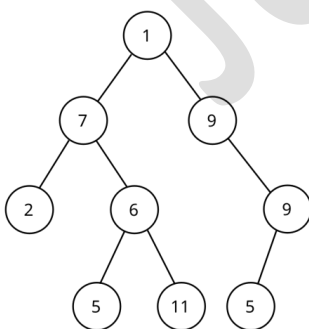
- Here, node 1 has 3 children, node 2 has 2 children.
- There is no restriction on the number of children.

2. Binary Tree

→ A tree in which each node has **at most two children** is called a **Binary Tree**. Children are generally called:

- Left Child
- Right Child

Example



- Every node has maximum 2 children.



Semester: 2

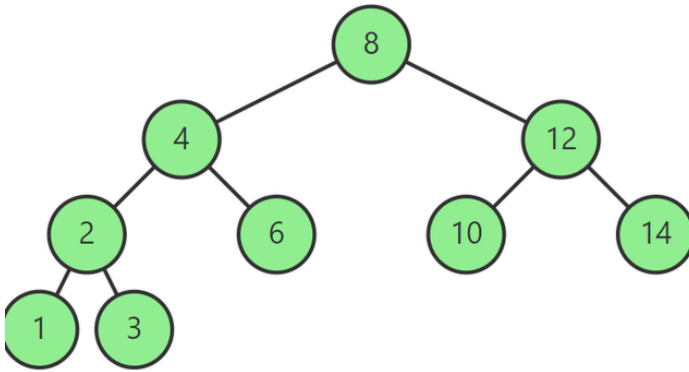
Subject : Data Structures using Java

Chapter : Tree

3. Balanced Tree

→ A tree in which the height difference between left and right subtrees is small (generally ≤ 1) is called a **Balanced Tree**.

Example



- Tree height is evenly distributed.
- Searching becomes faster.

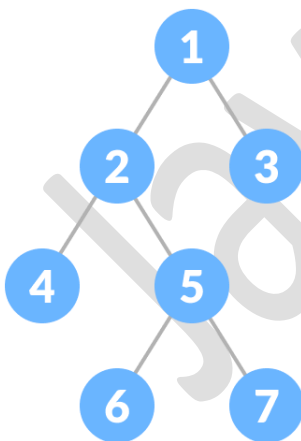
4. Full Binary Tree

→ A binary tree in which every node has either:

- 0 children, or
- 2 children

is called a **Full Binary Tree**.

Example



- 1 has 2 children
- 2, 5 has 2 children
- 3, 4, 6, 7 have 0 children
- No node has only one child.



Semester: 2

Subject : Data Structures using Java

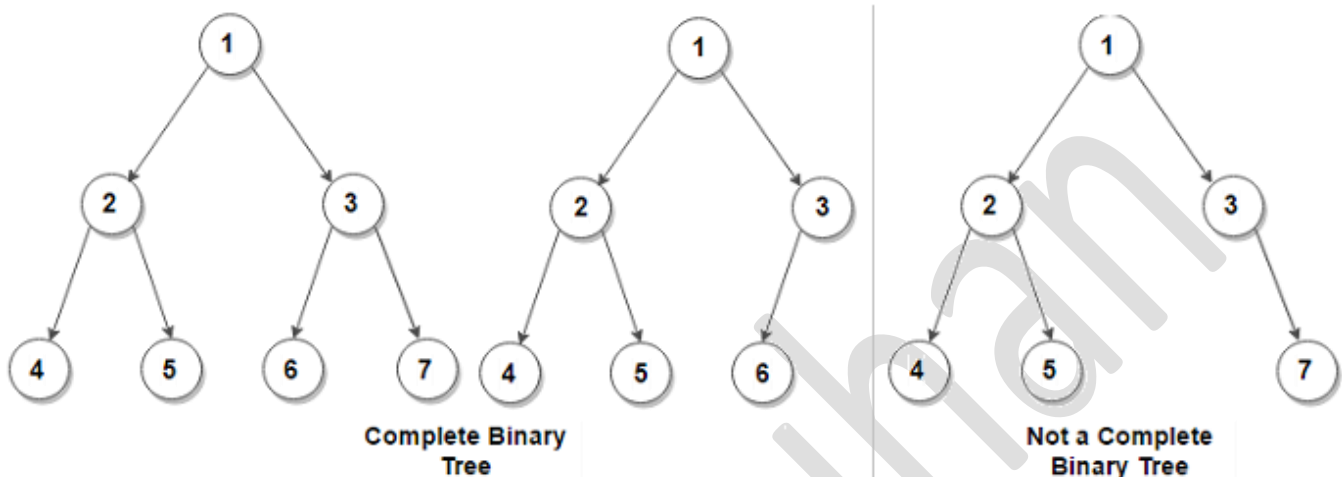
Chapter : Tree

5. Complete Binary Tree

→ A binary tree in which:

- All levels are completely filled except possibly the last level.
- Last level nodes are filled from left to right.

Example



- Last level is filled left to right.

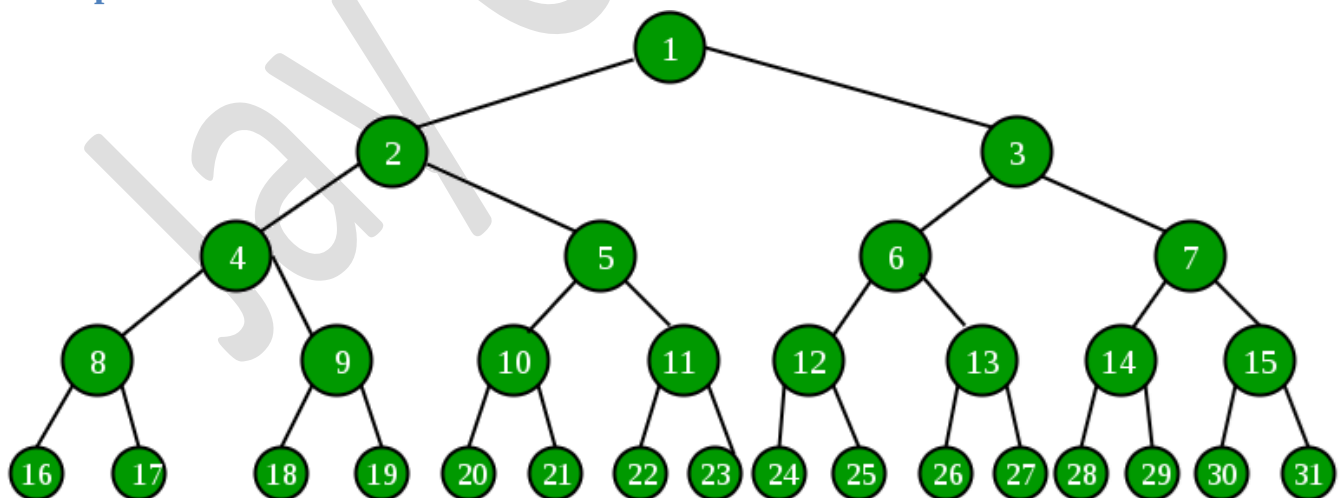
6. Perfect Binary Tree

→ A binary tree in which:

- All internal nodes have 2 children
- All leaf nodes are at the same level

is called a **Perfect Binary Tree**.

Example





Semester: 2

Subject : Data Structures using Java

Chapter : Tree

Properties:

- All levels completely filled
- Total nodes = $2^{h+1} - 1$

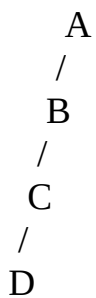
7. Skewed Tree

→ A tree in which all nodes are connected only to one side is called a **Skewed Tree**.

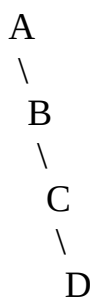
Types:

- Left Skewed
- Right Skewed

Left Skewed Tree



Right Skewed Tree

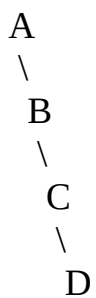


8. Degenerate or Pathological Tree

→ A tree in which every parent node has only one child is called a **Degenerate Tree** or **Pathological Tree**.

It behaves like a linked list.

Example



- Each node has only one child.
- Height becomes equal to number of nodes – 1.



Semester: 2

Subject : Data Structures using Java

Chapter : Tree

❖ Representation of Binary Tree

→ Binary Tree can be represented using:

1. **Array Representation**
2. **Linked List Representation**

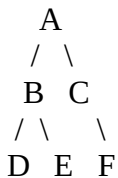
1. Array Representation of Binary Tree

→ In array representation, tree nodes are stored sequentially in an array.

For a node at index i :

- Left Child = $2i + 1$
- Right Child = $2i + 2$
- Parent = $(i - 1) / 2$

Example Binary Tree



Array Representation

Index	Value
0	A
1	B
2	C
3	D
4	E
5	NULL
6	F

Array Form:

[A, B, C, D, E, NULL, F]

→ Working of Indexes

For node A at index 0:

- Left Child → index $2(0)+1 = 1 \rightarrow B$
- Right Child → index $2(0)+2 = 2 \rightarrow C$

For node B at index 1:

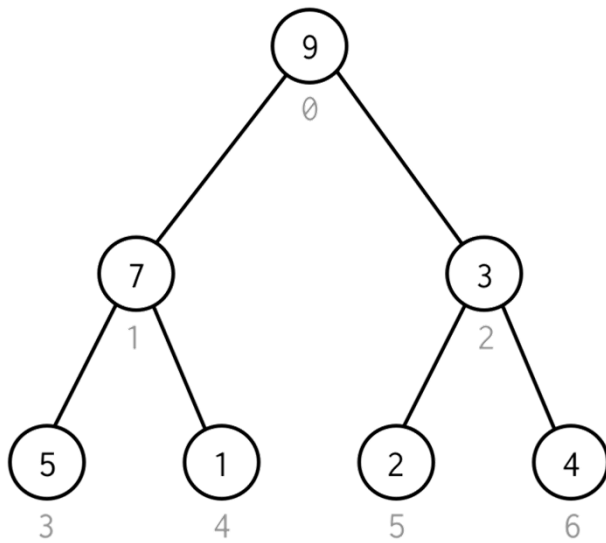
- Left Child → index $3 \rightarrow D$
- Right Child → index $4 \rightarrow E$



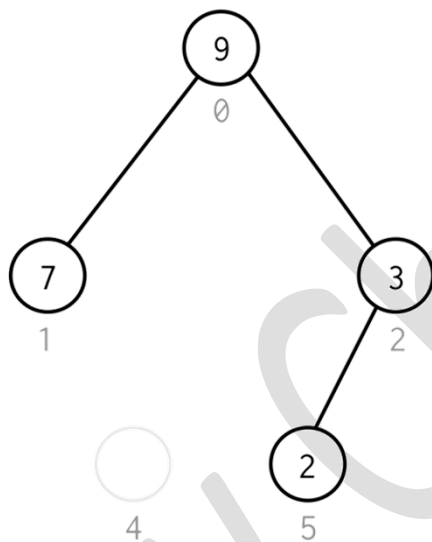
Semester: 2

Subject : Data Structures using Java

Chapter : Tree



9	7	3	5	1	2	4
0	1	2	3	4	5	6



9	7	3	null	null	2	null
0	1	2	3	4	5	6

➤ Advantages of Array Representation

- Easy implementation
- Fast access using indexes
- Efficient for complete binary trees

➤ Disadvantages of Array Representation

- Wastage of memory in sparse trees
- Fixed size array limitation



Semester: 2

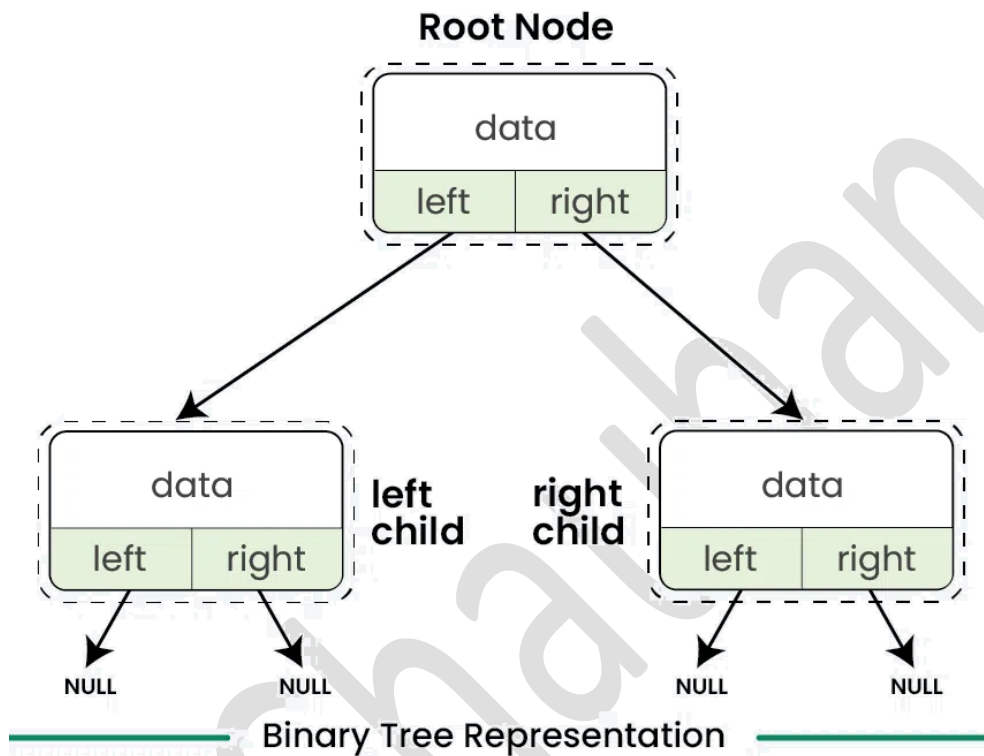
Subject : Data Structures using Java

Chapter : Tree

2. Linked List Representation of Binary Tree

→ In linked list representation, each node contains:

- Data
- Address of Left Child
- Address of Right Child



Structure of Node

| Left | Data | Right |

Example Binary Tree

```
A
 / \
B   C
 / \
D   E
```

Linked Representation

```
[A]
 / \
[B] [C]
 / \
[D] [E]
```

Each node stores links to its children.



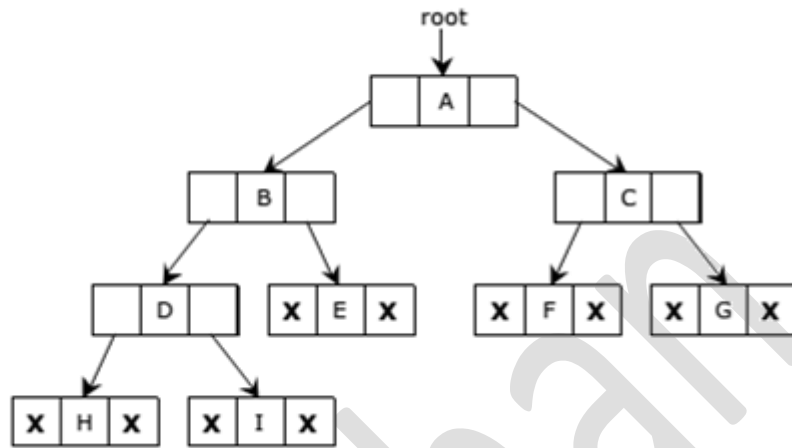
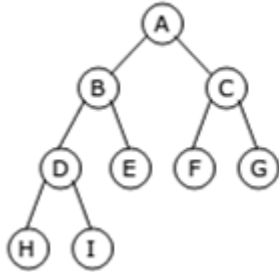
Semester: 2

Subject : Data Structures using Java

Chapter : Tree

Example:

- Node A stores address of B and C
- Node B stores address of D and E



Java Structure Example

```
class Node {  
    char data;  
    Node left;  
    Node right;  
}
```

Advantages of Linked List Representation

- Dynamic memory allocation
- No memory wastage
- Suitable for all tree types

Disadvantages of Linked List Representation

- Extra memory required for pointers
- Traversal is comparatively slower than arrays

❖ Difference Between Array and Linked Representation

Feature	Array Representation	Linked Representation
Memory Allocation	Static	Dynamic
Memory Usage	May waste memory	Efficient
Access Speed	Fast	Slower
Suitable For	Complete Trees	Sparse Trees
Implementation	Simple	More Complex



Semester: 2

Subject : Data Structures using Java

Chapter : Tree

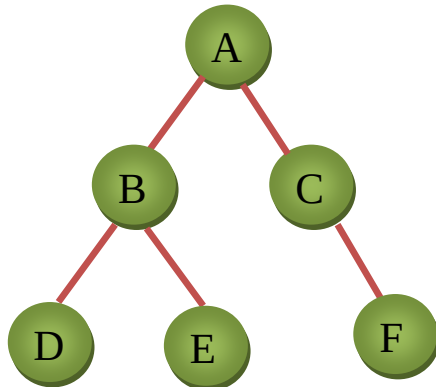
❖ Binary Tree Traversal

→ Traversal means visiting all nodes of a binary tree in a specific order.

Types of Binary Tree Traversal:

1. Preorder Traversal
2. Inorder Traversal
3. Postorder Traversal

Example Binary Tree



1. Preorder Traversal

Rule

Root → Left → Right

Steps

1. Visit Root node
2. Traverse Left Subtree
3. Traverse Right Subtree

Traversal of Given Tree

Output

A → B → D → E → C → F

Java Logic

```
void preorder(Node root)
{
    if(root != null)
    {
        System.out.print(root.data + " ");
        preorder(root.left);
        preorder(root.right);
    }
}
```



Semester: 2

Subject : Data Structures using Java

Chapter : Tree

2. Inorder Traversal

Rule

Left → Root → Right

Steps

1. Traverse Left Subtree
2. Visit Root node
3. Traverse Right Subtree

Traversal of Given Tree

```
  A
 /\
B  C
 /\  \
D E  F
```

Output

D → B → E → A → C → F

Java Logic

```
void inorder(Node root)
{
    if(root != null)
    {
        inorder(root.left);
        System.out.print(root.data + " ");
        inorder(root.right);
    }
}
```

3. Postorder Traversal

Rule

Left → Right → Root

Steps

1. Traverse Left Subtree
2. Traverse Right Subtree
3. Visit Root node

Traversal of Given Tree

```
  A
 /\
B  C
 /\  \
D E  F
```

Output

D → E → B → F → C → A

Java Logic

```
void postorder(Node root)
```



Semester: 2

Subject : Data Structures using Java

Chapter : Tree

```
{
    if(root != null)
    {
        postorder(root.left);
        postorder(root.right);
        System.out.print(root.data + " ");
    }
}
```

Difference Between Traversals

Traversal Type	Order
Preorder	Root → Left → Right
Inorder	Left → Root → Right
Postorder	Left → Right → Root

Memory Trick

Traversal	Shortcut
Preorder	NLR
Inorder	LNR
Postorder	LRN

Where:

- N = Node (Root)
- L = Left
- R = Right

❖ Binary Tree Traversal using Three Visit Method (Euler Tour Method)

- In Binary Tree traversal, each node is visited **three times** during recursion.
- To understand this easily:
- Assume every node has:
 - Left child
 - Right child
- If child is missing, consider it as a **Dummy (NULL) Child**.

➤ Three Visits of Node

During traversal:

Visit Number	Traversal Obtained
1st Visit	Preorder
2nd Visit	Inorder
3rd Visit	Postorder

Meaning of Visits

➤ 1st Visit

Occurs when control reaches the node for the first time.

→ Used in **Preorder Traversal**

Root → Left → Right



Semester: 2

Subject : Data Structures using Java

Chapter : Tree

➤ **2nd Visit**

Occurs after completing the left subtree and returning back to the node.

→ Used in **Inorder Traversal**

Left → Root → Right

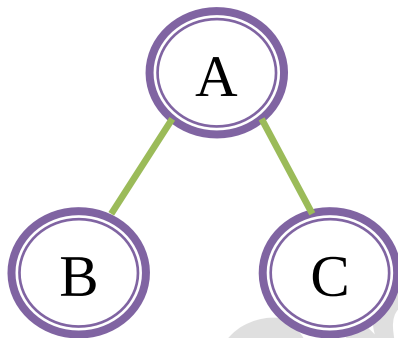
➤ **3rd Visit**

Occurs after completing both left and right subtrees.

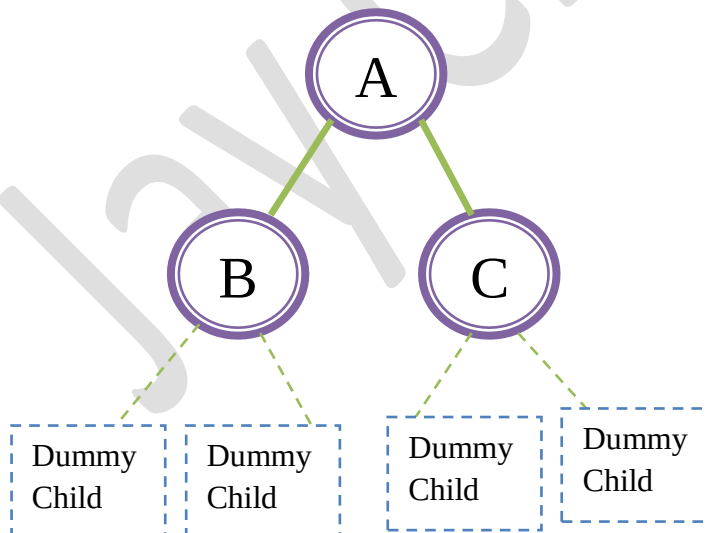
→ Used in **Postorder Traversal**

Left → Right → Root

Example :- Find the Pre-Order, In-Order & Post-Order Traversal of the given tree.



Step 1:- Make dummy children as necessary.



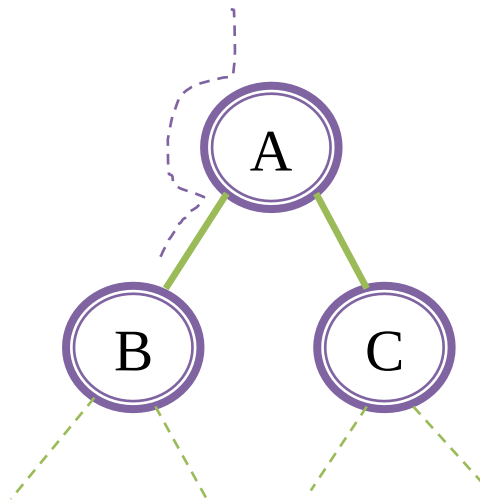


Semester: 2

Subject : Data Structures using Java

Chapter : Tree

Step 2: – Start from the Root Node.



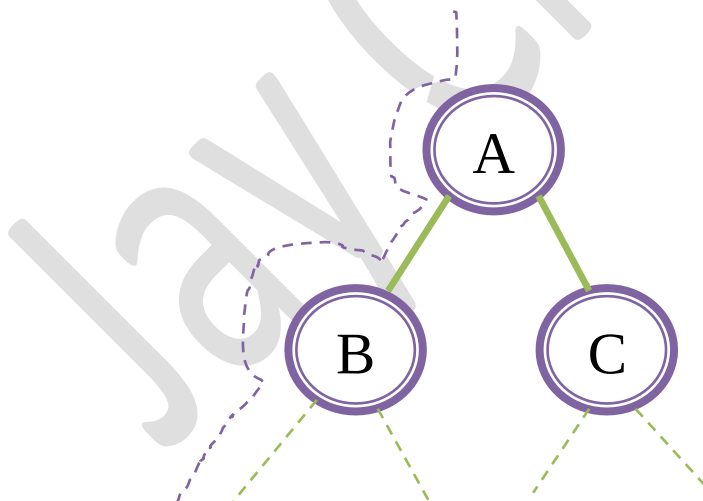
Here A is visited First Time So, We will write A in Pre-order.

Preorder :- A-

Inorder :-

Postorder:-

Step 3 – Start Traversing from Left to Right.



Here B is visited First Time So, We will write B in Pre-order.

Preorder :- A - B

Inorder :-

Postorder:-

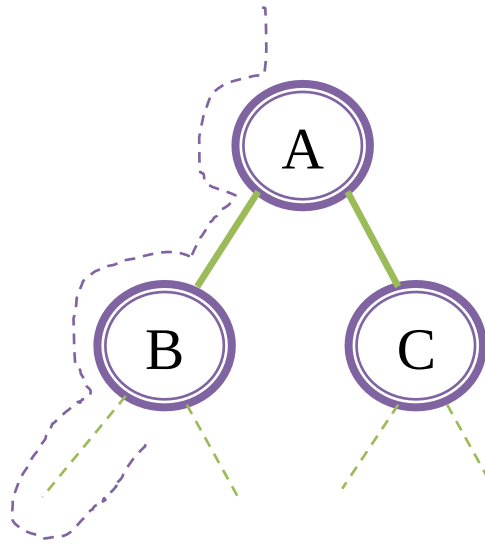


Semester: 2

Subject : Data Structures using Java

Chapter : Tree

Step 4 – Here We can see node B is visited 2nd time. so we will write in Inorder.

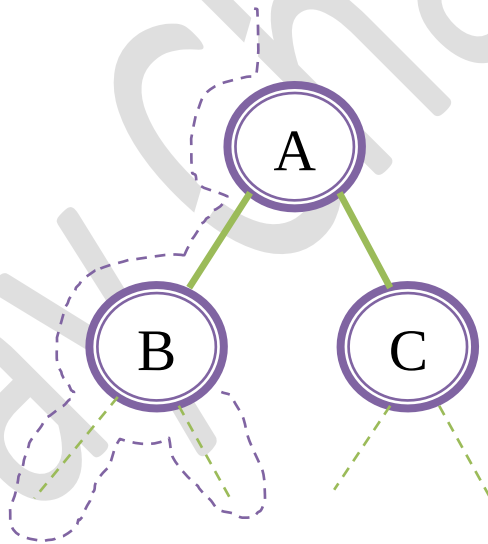


Preorder :- A - B

Inorder :- B

Postorder:-

Step 5 – Here We can see node B is visited 3rd time. so we will write in Postorder.



Preorder :- A - B

Inorder :- B

Postorder:- B

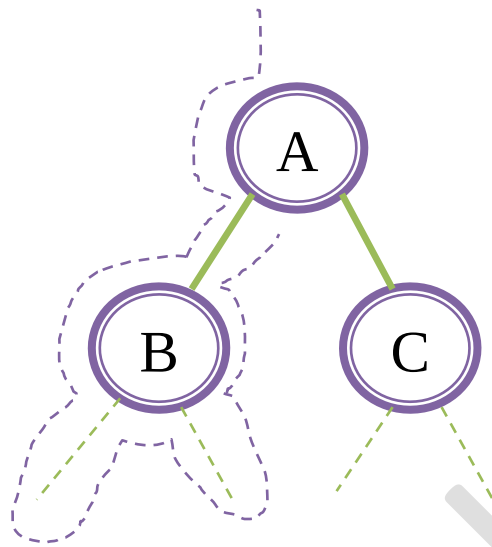


Semester: 2

Subject : Data Structures using Java

Chapter : Tree

Step 6 – Here We can see node A is visited 2nd time. so we will write in Inorder.

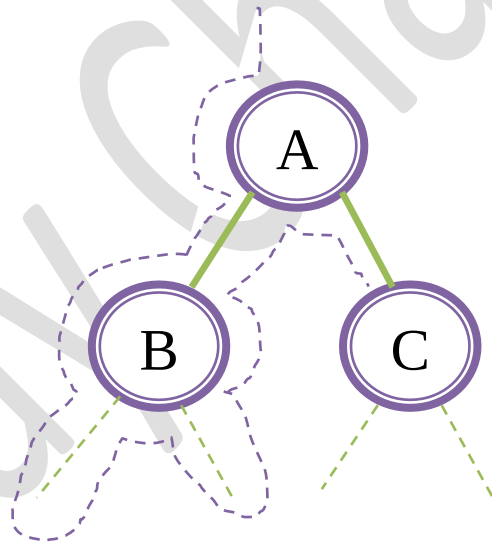


Preorder :- A - B

Inorder :- B - A

Postorder:- B

Step 7 – Here We can see node C is visited 1st time. so we will write in Preorder.



Preorder :- A – B - C

Inorder :- B - A

Postorder:- B

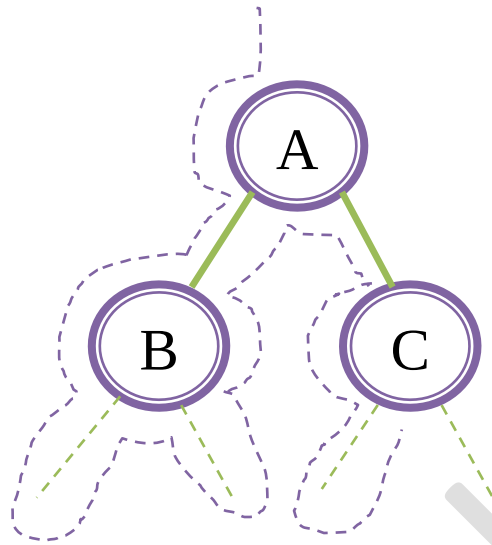


Semester: 2

Subject : Data Structures using Java

Chapter : Tree

Step 8 – Here We can see node C is visited 2nd time. so we will write in Inorder.

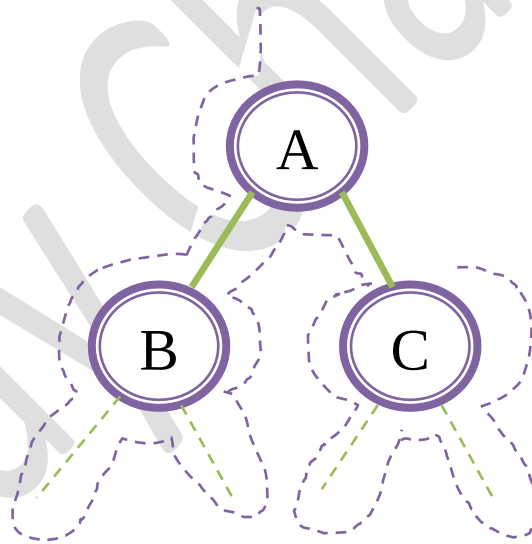


Preorder :- A – B – C

Inorder :- B – A – C

Postorder:- B

Step 9 – Here We can see node C is visited 3rd time. so we will write in Postorder.



Preorder :- A – B – C

Inorder :- B – A – C

Postorder:- B – C

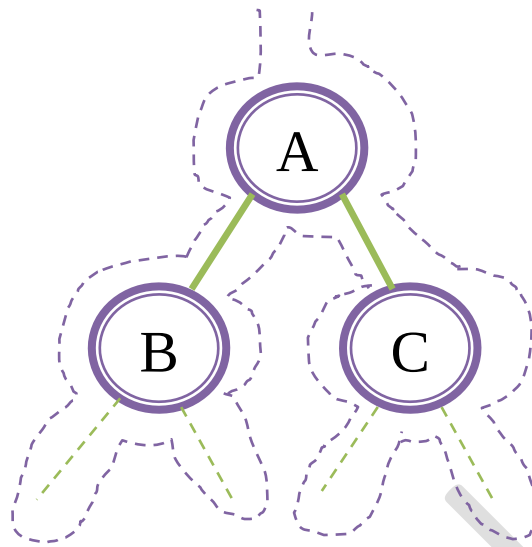


Semester: 2

Subject : Data Structures using Java

Chapter : Tree

Step 10 – Here We can see node A is visited 3rd time. so we will write in Postorder.



Preorder :- A – B – C

Inorder :- B – A – C

Postorder:- B – C – A

Important Points

- Every node is visited exactly 3 times.
- Dummy children are assumed where child nodes are missing.
- This method helps in:
 - Understanding recursion
 - Quickly solving traversal problems in exams
 - Visualizing recursive calls

Memory Trick

Visit	Traversal
1st Visit	Preorder
2nd Visit	Inorder
3rd Visit	Postorder



Semester: 2

Subject : Data Structures using Java

Chapter : Tree

❖ Construction of Binary Tree from Traversal

1. Inorder and preorder

Preorder: A B D G C E H I F

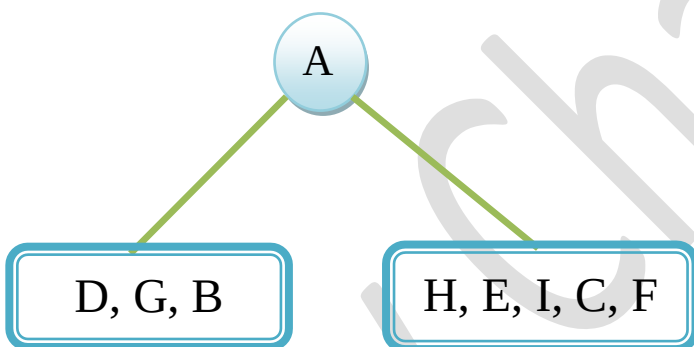
Inorder: D G B A H E I C F

→ Remember the definition of Pre-Order Traversal = Root – Left Child – Right Child.

Step 1:

- ✓ From Preorder sequence **A** B D G C E H I F, the root is: **A**
- ✓ From Inorder sequence D G B **A** H E I C F, we get the left and right sub trees:
 - Left sub tree is: D G B
 - Right sub tree is: H E I C F

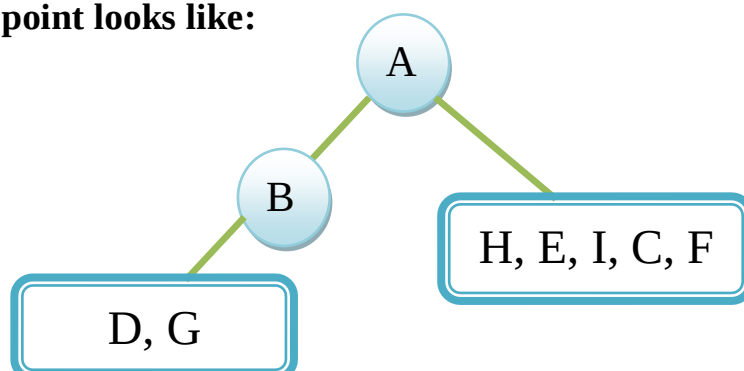
The Binary tree upto this point looks like:



Step 2:

- ✓ To find the root, left and right sub trees for D G B:
- ✓ From the preorder sequence **B** D G, the root of tree is: **B**
- ✓ From the inorder sequence D G **B**, we can find that D and G are to the left of B.

The Binary tree upto this point looks like:





Semester: 2

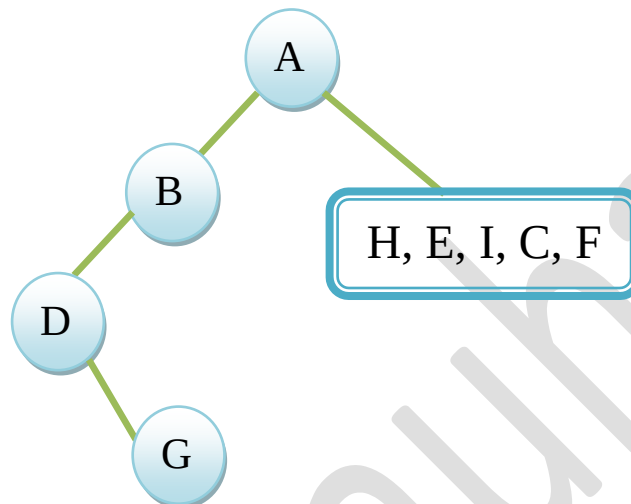
Subject : Data Structures using Java

Chapter : Tree

Step 3:

- ✓ To find the root, left and right sub trees for D G:
- ✓ From the preorder sequence **D** G, the root of the tree is: **D**
- ✓ From the inorder sequence **D** G, we can find that there is no left node to D and G is at the right of D.

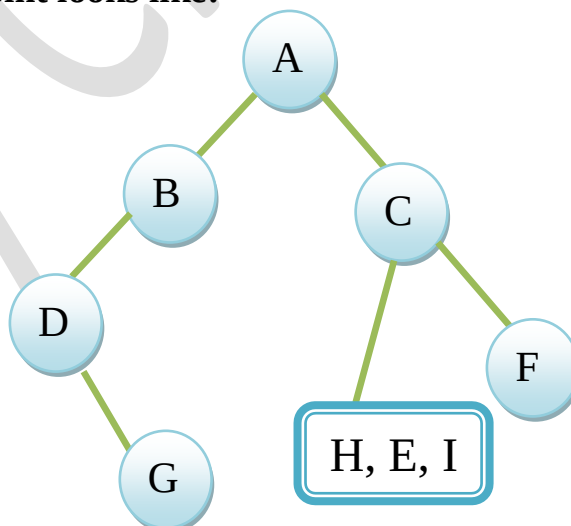
The Binary tree upto this point looks like:



Step 4:

- ✓ To find the root, left and right sub trees for H E I C F
- ✓ From the preorder sequence **C** E H I F, the root of the left sub tree is: **C**
- ✓ From the inorder sequence H E I **C** F, we can find that H E I are at the left of C and F is at the right of C.

The Binary tree upto this point looks like:





Semester: 2

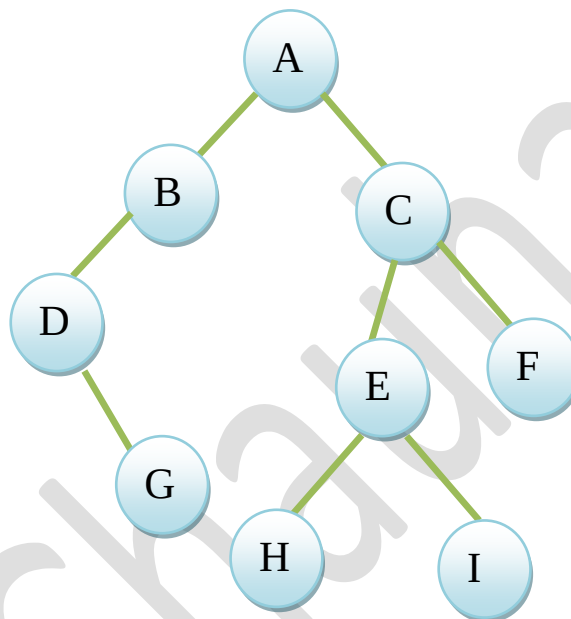
Subject : Data Structures using Java

Chapter : Tree

Step 5:

- ✓ To find the root, left and right sub trees for H E I:
- ✓ From the preorder sequence E H I, the root of the tree is: **E**
- ✓ From the inorder sequence H **E** I, we can find that H is at the left of E and I is at the right of E.

The Binary tree upto this point looks like:



2. Inorder and Postorder

Post order: I-D-B-G-C-H-F-E-A

Inorder: B-I-D-A-C-G-E-H-F

- Remember the definition of Post-Order Traversal = Left Child – Right Child - Root.
- Hence, the Last value from the given Post-Order Traversal will be the Root of the binary tree. Here in our case, it is A.

Step 1:

- ✓ Root of the Binary Tree (Last value of the Pre-Order Traversal) = A.
- ✓ Post order: I-D-B-G-C-H-F-E-**A**
- ✓ Inorder: B-I-D-**A**-C-G-E-H-F



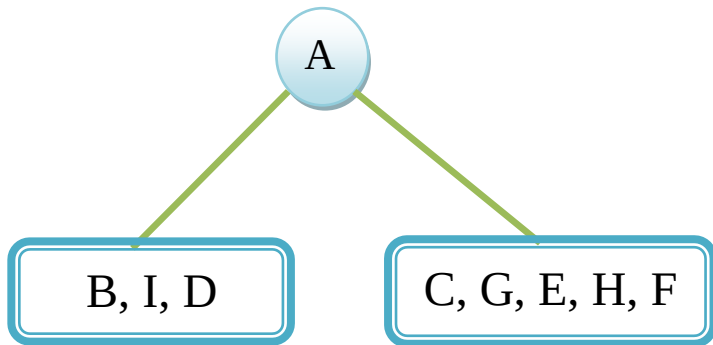
Semester: 2

Subject : Data Structures using Java

Chapter : Tree

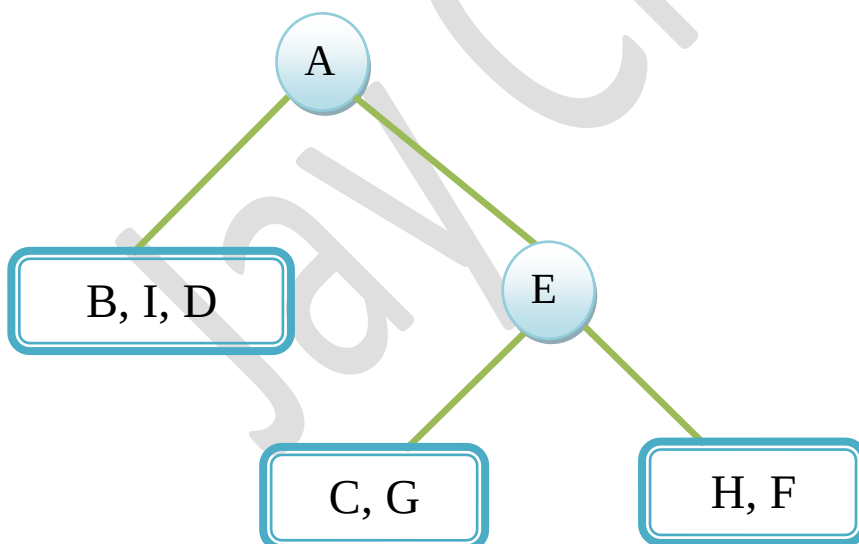
- ✓ Now, highlight the same value in In-Order Traversal. As you can see above, when I highlighted A in In-Order traversal (B-I-D) are on the left side of tree and (C-G-E-H-F) is on the right side of the tree.

And this will hold true when we construct the tree.



Step 2 :

- ✓ Now, again see the Post-Order Traversal to get the next root. Here in our case, it is E.
- ✓ Post order: I-D-B-G-C-H-F-**E**-**A**
- ✓ Inorder: B-I-D-**A**-C-G-**E**-H-F
- ✓ As you can see in step 1, the value E is already on the right of the Node A. Hence (C-G) will be on the left of E and (H-F) will be on the right of E.





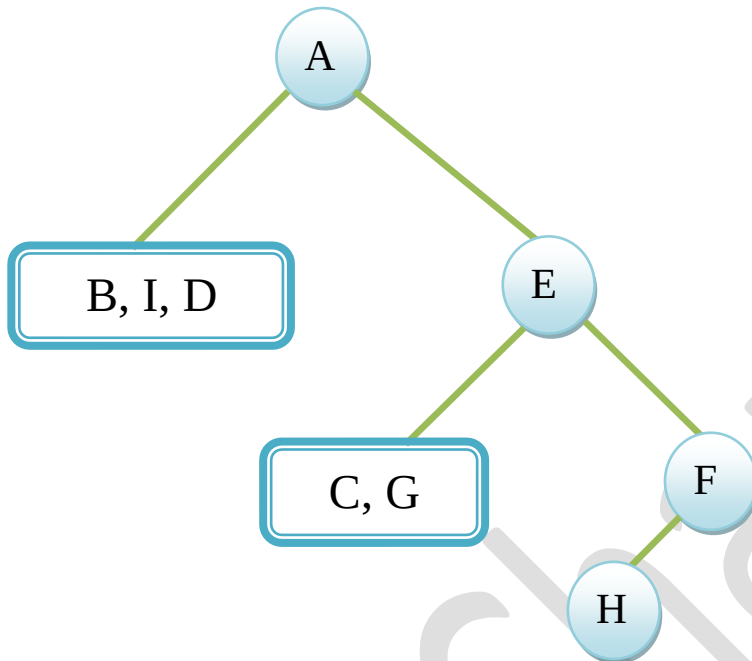
Semester: 2

Subject : Data Structures using Java

Chapter : Tree

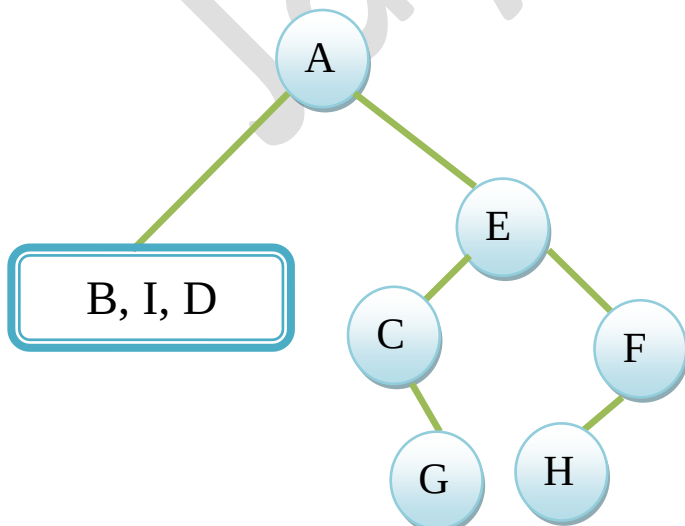
Step 3 :

- ✓ Now, again see the Post-Order Traversal to get the next root. Here in our case, it is F. & also you see the Tree in step 2. H is on the left of F.
- ✓ Post order: I-D-B-G-C-H-F-E-A
- ✓ Inorder: B-I-D-A-C-G-E-H-F



Step 4 :

- ✓ Now, again see the Post-Order Traversal to get the next root. Here in our case, it is C. & also you see the Tree in step 3. G is on the right of C.
- ✓ Post order: I-D-B-G-C-H-F-E-A
- ✓ Inorder: B-I-D-A-C-G-E-H-F





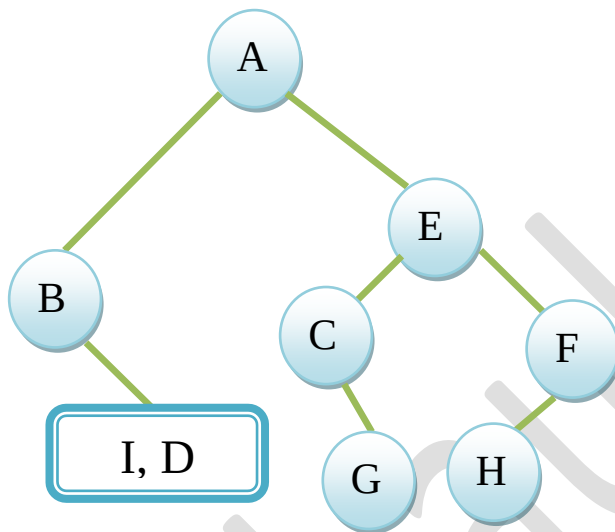
Semester: 2

Subject : Data Structures using Java

Chapter : Tree

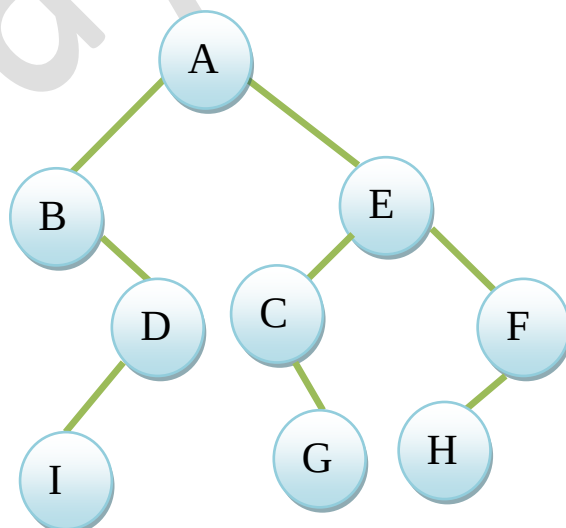
Step 5 :

- ✓ Now, again see the Post-Order Traversal to get the next root. Here in our case, it is B. So, when we will make B the root, I-D both will be on the right of B as seen in In-Order.
- ✓ Post order: I-D-B-G-C-H-F-E-A
- ✓ Inorder: B-I-D-A-C-G-E-H-F



Step 6 :

- ✓ Now, again see the Post-Order Traversal to get the next root. Here in our case, it is D. So, when we will make D the root, I will be on the left of D as seen in In-Order.
- ✓ Post order: I-D-B-G-C-H-F-E-A
- ✓ Inorder: B-I-D-A-C-G-E-H-F





Semester: 2

Subject : Data Structures using Java

Chapter : Tree

CONSTRUCT FULL BINARY TREE FROM PREORDER AND POSTORDER

Given:

Preorder (Pre) = { 1, 2, 4, 8, 9, 5, 3, 6, 7 }

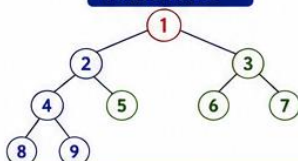
Postorder (Post) = { 8, 9, 4, 5, 2, 6, 7, 3, 1 }

Note:

In Full Binary Tree,
every node has either
0 or 2 children.

Step	Explanation	Preorder (Pre)	Postorder (Post)	Tree after this step
1 Select Root	- The first element of Preorder is the root. - Also, last element of Postorder is the root. Root = 1	1, 2, 4, 8, 9, 5, 3, 6, 7	8, 9, 4, 5, 2, 6, 7, 3, 1	
2 Find Left Subtree	- Next element in Preorder after 1 is 2, So, 2 is the root of left subtree. - Find 2 in Postorder. - Elements up to and including 2 form the left subtree. - Remaining elements form right subtree.	1 2, 4, 8, 9, 5 3, 6, 7	8, 9, 4, 5, 2 6, 7, 3 1	
3 Construct Left Subtree of 1	Left Subtree: Preorder : 2, 4, 8, 9, 5 Postorder : 8, 9, 4, 5, 2 - Root = 2 - Next element in Preorder is 4 → left child of 2. - Remaining element 5 → right child of 2.	2, 4, 8, 9, 5	8, 9, 4, 5, 2	
4 Construct Subtree of 4	For node 4: Preorder : 4, 8, 9 Postorder : 8, 9, 4 So, 8 and 9 are children of 4.	4, 8, 9	8, 9, 4	
5 Construct Right Subtree of 1	Right Subtree: Preorder : 3, 6, 7 Postorder : 6, 7, 3 - Root = 3 - Next element in Preorder is 6 → left child of 3. - Remaining element 7 → right child of 3.	3, 6, 7	6, 7, 3	
6 Final Tree (Completed)	Both left and right subtrees are constructed. This is the required full binary tree.	—	—	

Final Answer



Verification

Preorder (Root - Left - Right) : 1, 2, 4, 8, 9, 5, 3, 6, 7 ✓

Postorder (Left - Right - Root) : 8, 9, 4, 5, 2, 6, 7, 3, 1 ✓

Inorder (Left - Root - Right) : 8, 4, 9, 2, 5, 1, 6, 3, 7



Thus, the Full Binary Tree constructed from the given Preorder and Postorder traversals is as shown above.



Semester: 2

Subject : Data Structures using Java

Chapter : Tree

CONSTRUCT BINARY TREE FROM PREORDER AND POSTORDER TRAVERSALS

Given:

Preorder : A, B, D, C, E, G, F, H, I

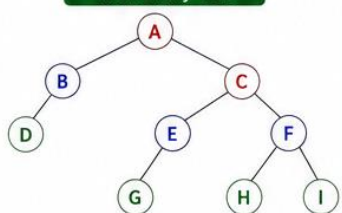
Postorder : D, B, G, E, H, I, F, C, A

Note:

A binary tree is uniquely determined from Preorder and Postorder if it is a Full Binary Tree.

Step	Explanation	Preorder	Postorder	Left Subtree	Right Subtree	Tree after this step
1 Find Root	- First element of Preorder = A - Last element of Postorder = A Root = A	A, B, D, C, E, G, F, H, I	D, B, G, E, H, I, F, C, A	—	—	
2 Find Left and Right Subtree of A	- Next element after A in Preorder is B. So, B is left child of A. - Find B in Postorder. - Elements up to and including B belong to left subtree. - Remaining elements belong to right subtree.	A, B, D C, E, G, F, H, I	D, B G, E, H, I, F, C A	B, D	C, E, G, F, H, I	
3 Construct Left Subtree (Rooted at B)	Left Subtree: Preorder : B, D Postorder : D, B - Root = B - Next element after B is D. So, D is left child of B.	B, D	D, B	—	—	
4 Construct Right Subtree (Rooted at C)	Right Subtree: Preorder : C, E, G, F, H, I Postorder : G, E, H, I, F, C - Root = C - Next element after C is E. So, E is left child of C. - Remaining part gives F as right child of C.	C, E, G, F, H, I	G, E, H, I, F, C	E, G	F, H, I	
5 Construct Subtrees of E and F	For E: Preorder : E, G Postorder : G, E - So, G is left child of E. For F: Preorder : F, H, I Postorder : H, I, F - So, H is left child of F and I is right child of F.	E, G	G, E	G	—	
6 Final Tree (Completed)	- Both left and right subtrees are constructed. - This is the required binary tree.	—	—	—	—	

Final Binary Tree



Verification

Preorder (Root – Left – Right)
= A, B, D, C, E, G, F, H, I ✓

Postorder (Left – Right – Root)
= D, B, G, E, H, I, F, C, A ✓

Answers

1. Height of the Tree: **3**

2. Elements at Level 2: **D, E, F**

Level 0 → **A**

Level 1 → **B** **C**

Level 2 → **D** **E** **F**

Level 3 → **G** **H** **I**

★ Thus, the binary tree is constructed from the given Preorder and Postorder traversals.



Semester: 2

Subject : Data Structures using Java

Chapter : Tree

❖ Conversion from General Tree to Binary Tree

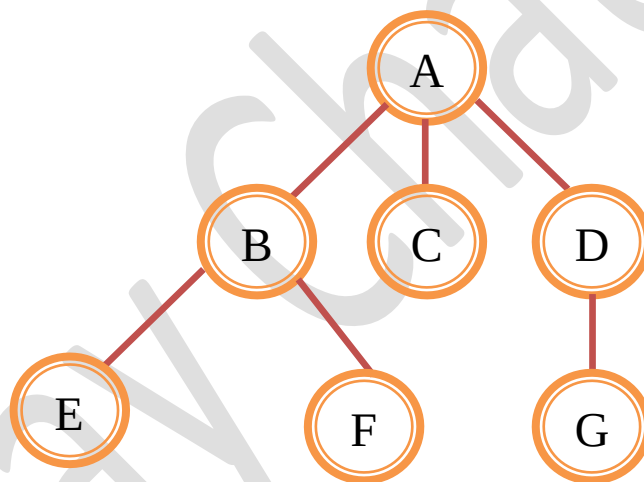
→ Converting a general tree (where a node can have any number of children) to a binary tree (at most two children) is standardly achieved using the Left-Child Right-Sibling (LCRS) representation. This method preserves the hierarchical relationships of the original tree while conforming to binary constraints.

- The Three Golden Rules

→ The conversion follows three primary rules for every node:

- **Root Rule:** The root of the general tree remains the root of the binary tree.
- **Left Child Rule:** The leftmost child of a node in the general tree becomes its left child in the binary tree.
- **Right Child Rule:** The immediate right sibling of a node in the general tree becomes its right child in the binary tree.

Example:



Step-by-Step Conversion

- **Root Rule:** The root of the general tree remains the root of the binary tree. i.e. A is root of Binary tree.
- **Left Child Rule:** The leftmost child of a node in the general tree becomes its left child in the binary tree.
 - B becomes left child of A
 - E becomes left child of B
 - G becomes left child of D

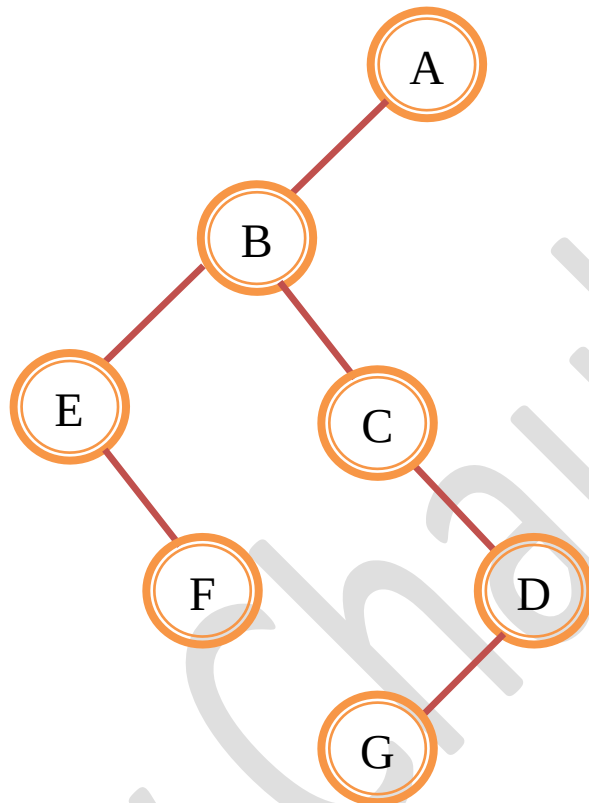


Semester: 2

Subject : Data Structures using Java

Chapter : Tree

- **Right Child Rule:** The immediate right sibling of a node in the general tree becomes its right child in the binary tree.
 - C becomes right child of B
 - D becomes right child of C
 - F becomes right child of E





Semester: 2

Subject : Data Structures using Java

Chapter : Tree

❖ Threaded Binary Tree

Quantity	Threaded Binary Tree
Definition	A binary tree in which NULL pointers are replaced with special links called threads
Purpose	Faster traversal without recursion or stack
Thread	Link to inorder predecessor or inorder successor
Types	Left Threaded, Right Threaded, Fully Threaded
Left Threaded Tree	NULL left pointer points to inorder predecessor
Right Threaded Tree	NULL right pointer points to inorder successor
Fully Threaded Tree	Both left and right NULL pointers are replaced by threads
Advantage	Traversal becomes easy and memory efficient
Disadvantage	Insertion and deletion become complex

❖ Types of Threaded Binary Tree

Type	Meaning
Left Threaded Binary Tree	Left NULL link stores inorder predecessor
Right Threaded Binary Tree	Right NULL link stores inorder successor
Fully / Double Threaded Binary Tree	Both predecessor and successor links are stored

❖ Important Point

→ Threaded Binary Tree is mainly used for inorder traversal without recursion.

❖ Comparison Table

Feature	Normal Binary Tree	Threaded Binary Tree
NULL links	Wasted	Used as threads
Traversal	Stack/Recursion needed	No stack required
Memory usage	More	Efficient
Traversal speed	Slower	Faster

❖ Left Threaded Binary Tree

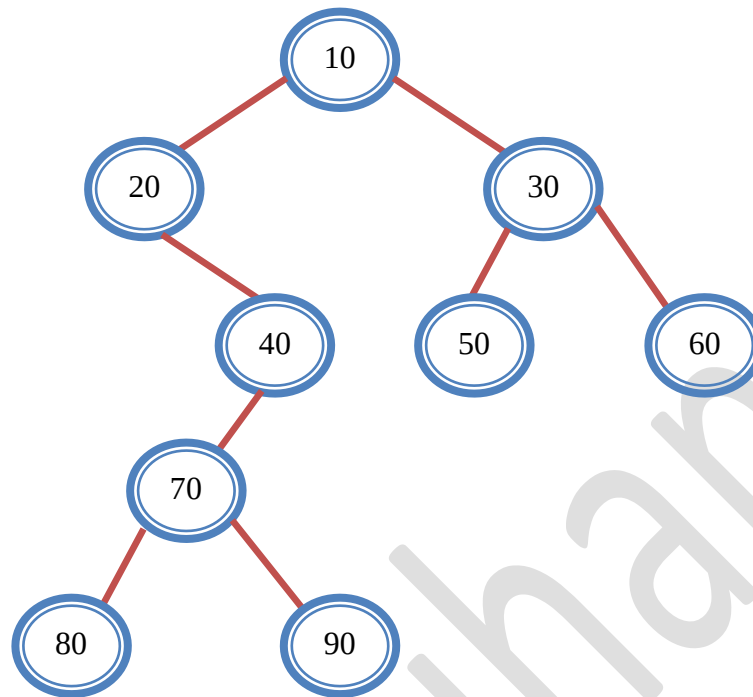
Point	Theory
Definition	A binary tree where NULL left pointers are replaced by threads
Thread Direction	Points to inorder predecessor
Purpose	Easy inorder traversal without recursion
Right Pointer	Remains normal child link
When Thread is Created	Only when left child is NULL
If Left Child Already Connected	No thread is required
Main Use	To move toward previous inorder node easily



Semester: 2

Subject : Data Structures using Java

Chapter : Tree



Inorder:- 20-80-70-90-40-10-50-30-60

Rule

If left child is NULL,
connect it with inorder predecessor.

Stepwise Explanation

Node	Left Child Present?	Predecessor	Left Thread
20	No	None	Header / NULL
80	No	20	80 → 20
70	Yes	80	No thread
90	No	70	90 → 70
40	Yes	90	No thread
10	Yes	40	No thread
50	No	10	50 → 10
30	Yes	50	No thread
60	No	30	60 → 30

Final Left Threads

20 → Header/NULL

80 → 20

90 → 70

50 → 10

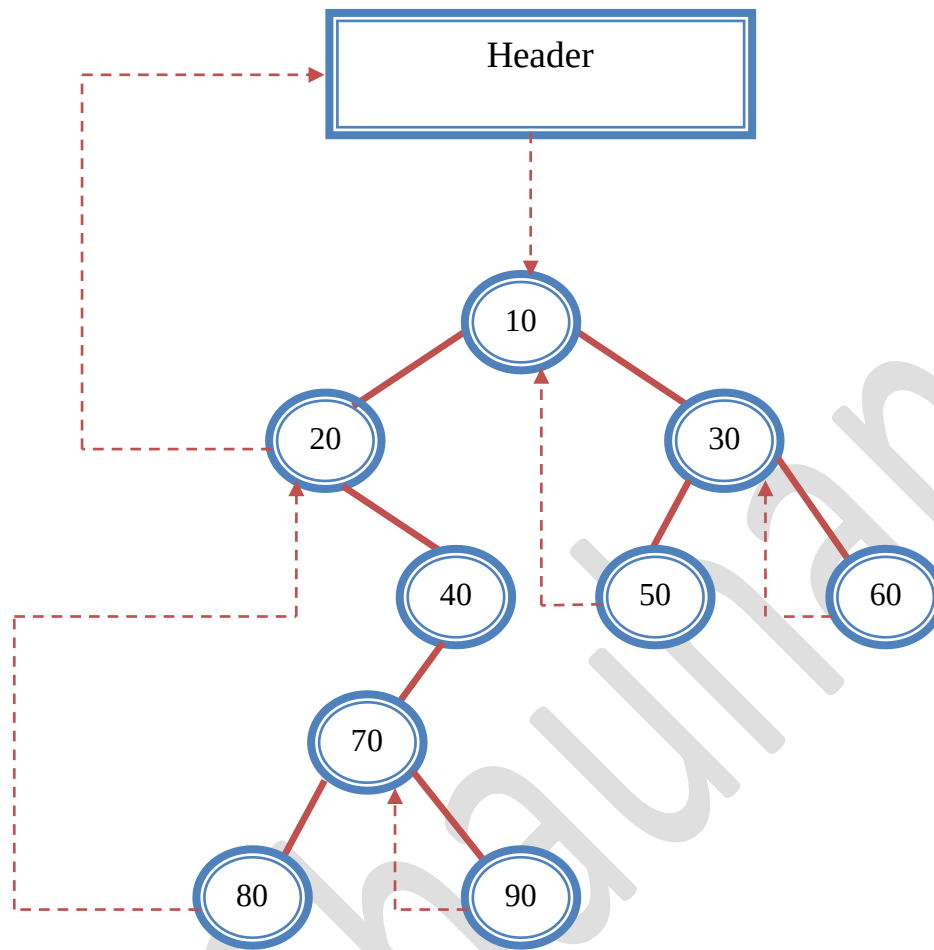
60 → 30



Semester: 2

Subject : Data Structures using Java

Chapter : Tree



❖ Right Threaded Binary Tree

Point	Theory
Definition	A binary tree where NULL right pointers are replaced by threads
Thread Direction	Points to inorder successor
Purpose	Easy inorder traversal without stack
Left Pointer	Remains normal child link
When Thread is Created	Only when right child is NULL
If Right Child Already Connected	No thread is required
Main Use	To move toward next inorder node easily

Rule

If right child is NULL,
connect it with inorder successor.



Semester: 2

Subject : Data Structures using Java

Chapter : Tree

Stepwise Explanation

Node	Right Child Present?	Successor	Right Thread
20	Yes (80)	80	20 → 80
80	No	70	80 → 70
70	Yes	90	No thread
90	No	40	90 → 40
40	No	10	40 → 10
10	Yes	50	No thread
50	No	30	50 → 30
30	Yes	60	No thread
60	No	None	60 → Header/NULL

Final Right Threads

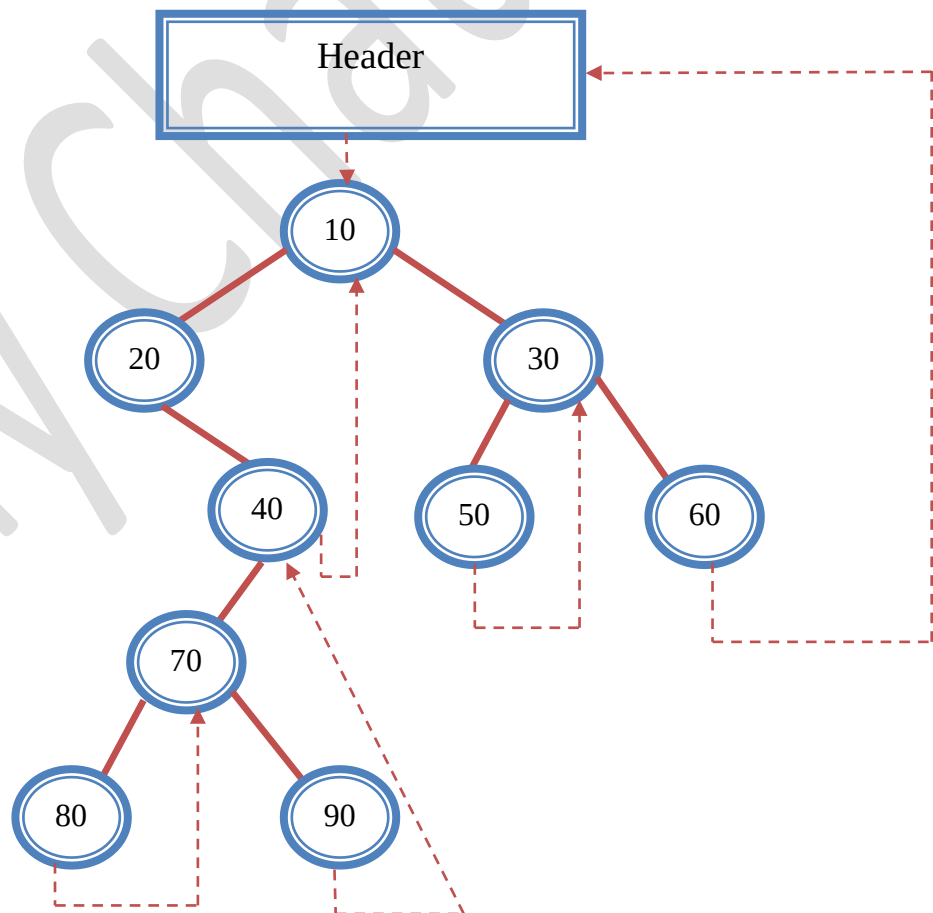
80 → 70

90 → 40

40 → 10

50 → 30

60 → Header/NULL



Inorder:- 20-80-70-90-40-10-50-30-60



Semester: 2

Subject : Data Structures using Java

Chapter : Tree

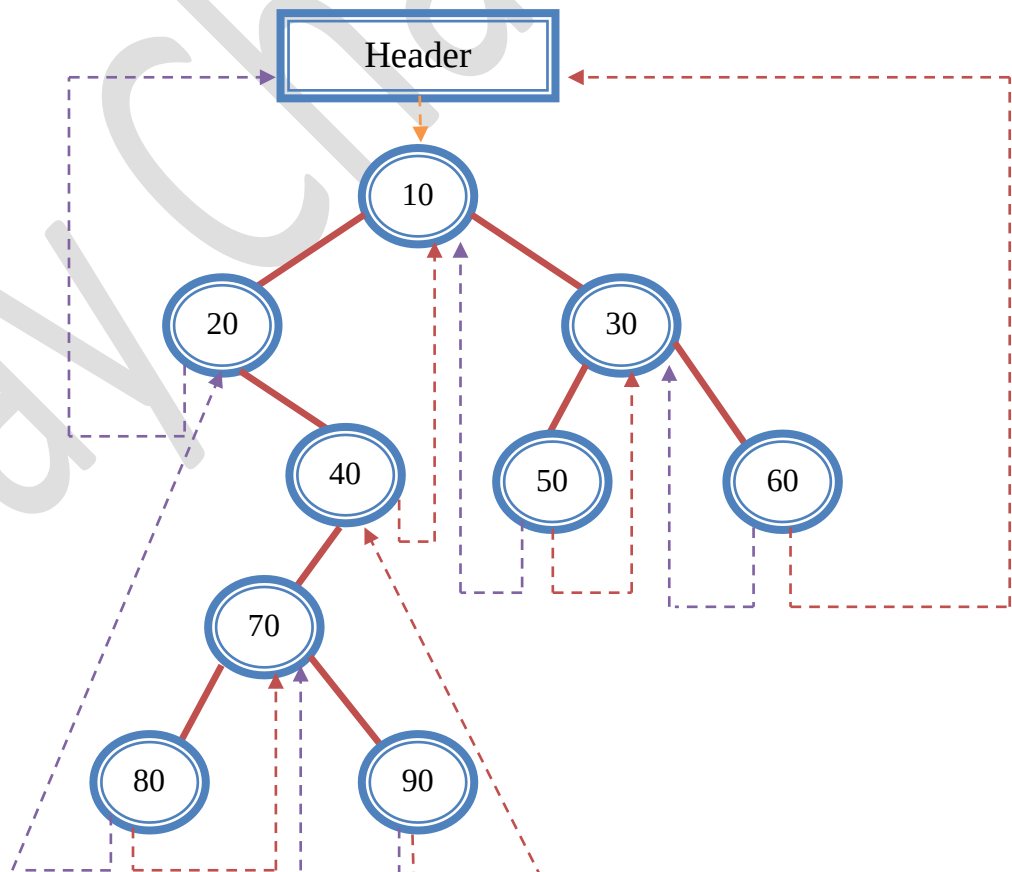
❖ Fully / Double Threaded Binary Tree

Point	Theory
Definition	A binary tree where both NULL left and NULL right pointers are replaced by threads
Left Thread	Points to inorder predecessor
Right Thread	Points to inorder successor
Purpose	Complete inorder traversal without recursion or stack
When Thread is Created	When left/right child is NULL
If Child Already Connected	No thread is required
Also Called	Double Threaded Binary Tree

Rule

Apply both left and right threading.

Left Threads	Right Threads
20 → Header/NULL	80 → 70
80 → 20	90 → 40
90 → 70	40 → 10
50 → 10	50 → 30
60 → 30	60 → Header/NULL



Inorder:- 20-80-70-90-40-10-50-30-60



Semester: 2

Subject : Data Structures using Java

Chapter : Tree

❖ How to Create Threads

Step	Description
Step 1	Find inorder traversal of tree
Step 2	Check all NULL left/right pointers
Step 3	Replace NULL left pointer with inorder predecessor
Step 4	Replace NULL right pointer with inorder successor
Step 5	Mark those special links as threads

❖ Important Rule

Condition	Action
Child pointer already connected to node	Do NOT create thread
Child pointer is NULL	Create thread
Left NULL pointer	Connect to inorder predecessor
Right NULL pointer	Connect to inorder successor

❖ Shortcut Line

Threads are created only on NULL pointers.
If a child already exists, no thread is added.

❖ Linked Representation of Threaded Binary Tree

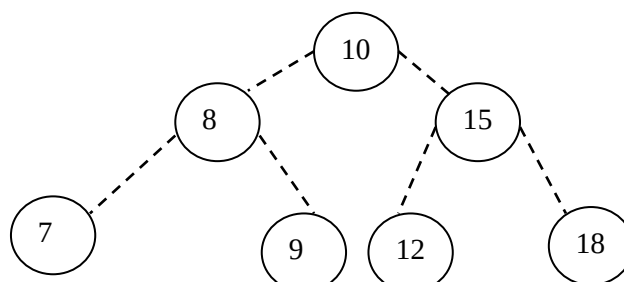
Each node contains:

Field	Meaning
Left Pointer	Points to left child or inorder predecessor thread
Left Tag (LTAG)	Identifies child/thread on left side
Data	Stores node value
Right Tag (RTAG)	Identifies child/thread on right side
Right Pointer	Points to right child or inorder successor thread

Structure

| LEFT | LTAG | DATA | RTAG | RIGHT |

→ Tree



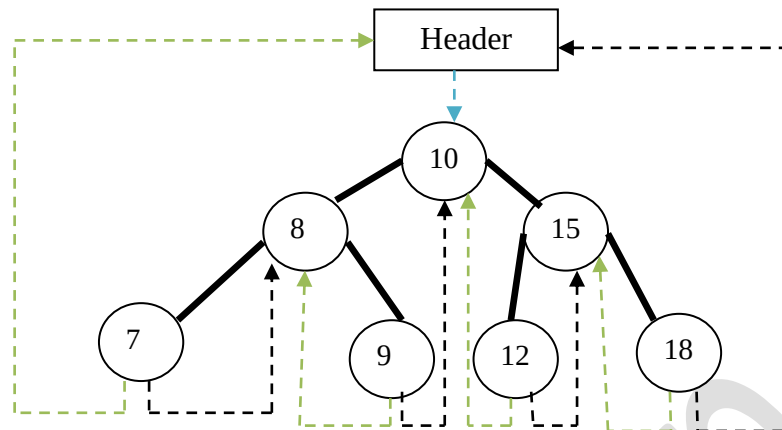


Semester: 2

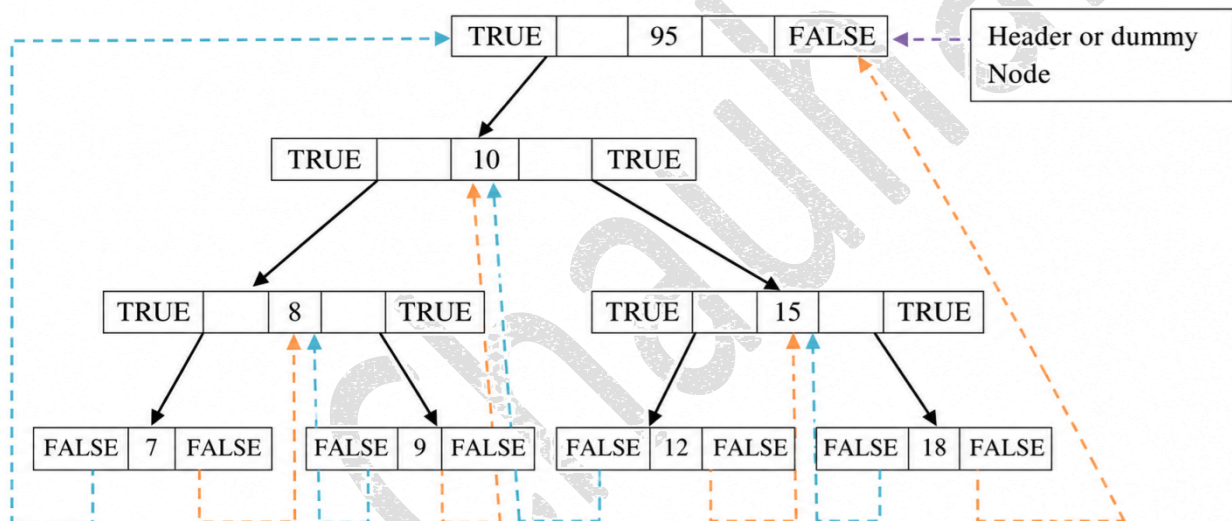
Subject : Data Structures using Java

Chapter : Tree

Full Threaded Tree



Inorder:- 7-8-9-10-12-15-18





Semester: 2

Subject : Data Structures using Java

Chapter : Tree

❖ Expression Tree

→ An **Expression Tree** is a binary tree used to represent arithmetic expressions.

Rules of Expression Tree

Node Type	Stored Value
Internal Node	Operator (+, -, *, /, ^)
Leaf Node	Operand (A, B, 5, X)

➤ Important Property

- Operators are always parent nodes.
- Operands are always leaf nodes.

➤ Example

Expression:

A + B

Expression Tree:

```
  +
 / \
A   B
```

Another Example

Expression:

A + B * C

Tree:

```
  +
 / \
A   *
    / \
   B  C
```

Construction Rules

Step	Description
1	Find operator
2	Make it root
3	Left expression becomes left subtree
4	Right expression becomes right subtree
5	Repeat recursively



Semester: 2

Subject : Data Structures using Java

Chapter : Tree

Operator Priority

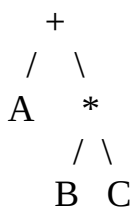
Priority	Operators
Highest	$\wedge$ \$ $\uparrow$
Medium	$*$ / $\%$
Lowest	$+$ -

Traversal Meaning

Traversal	Expression Form
Preorder	Prefix
Inorder	Infix
Postorder	Postfix

Example

Expression Tree:



Traversal	Output
Preorder	+ A * B C
Inorder	A + B * C
Postorder	A B C * +

Prefix / Infix / Postfix Relation

Expression Type	Example
Infix	A + B
Prefix	+ A B
Postfix	A B +

Applications of Expression Tree

Use	Purpose
Compiler Design	Expression evaluation
Calculator	Arithmetic operations
Parsing	Syntax analysis
Conversion	Prefix/Infix/Postfix conversion

Shortcut

Root = operator

Leaves = operands



Semester: 2

Subject : Data Structures using Java

Chapter : Tree

EXPRESSION TREE FROM PREFIX

RULE: Scan the prefix expression from **RIGHT** to **LEFT**.

STEPS:

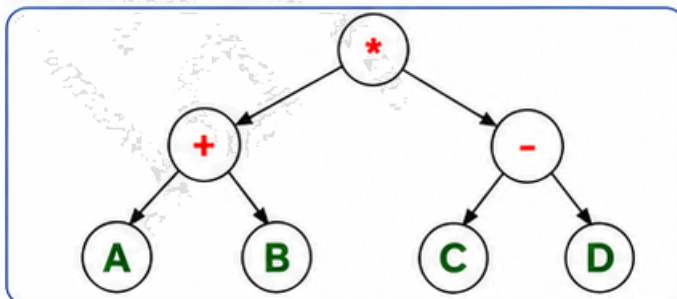
- 1 If operand → Create node and push into stack.
- 2 If operator → Pop two nodes.
- 3 First popped node becomes **LEFT** child.
- 4 Second popped node becomes **RIGHT** child.
- 5 Create operator node and push back into stack.
- 6 Repeat until expression ends.
- 7 Remaining stack node is the **ROOT** of the tree.

EXAMPLE:

Prefix Expression: ***+AB-CD**

Step	Symbol	Action	Stack (Top → Bottom)
1	D	Operand → Push	D
2	C	Operand → Push	C D
3	-	Operator → Pop C, D Make '-' (C = Left, D = Right)	-
4	B	Operand → Push	B -
5	A	Operand → Push	A B -
6	+	Operator → Pop A, B Make '+' (A = Left, B = Right)	+ -
7	*	Operator → Pop (+), (-) Make '*' ((+)=Left, (-)=Right)	*

EXPRESSION TREE:



Legend:

Red → Operator

Green → Operand

QUICK MEMORY TRICK

PREFIX

Scan: Right to Left

Operator:

First Pop → **Left** Child

Second Pop → **Right** Child



Semester: 2

Subject : Data Structures using Java

Chapter : Tree

EXPRESSION TREE FROM POSTFIX

RULE: Scan the postfix expression from **LEFT** to **RIGHT**.

STEPS:

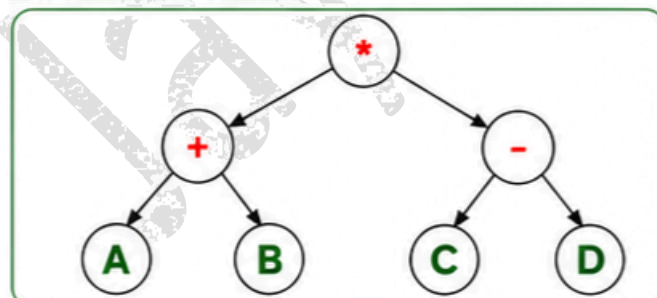
- 1 If operand → Create node and push into stack.
- 2 If operator → Pop two nodes.
- 3 First popped node becomes **RIGHT** child.
- 4 Second popped node becomes **LEFT** child.
- 5 Create operator node and push back into stack.
- 6 Repeat until expression ends.
- 7 Remaining stack node is the **ROOT** of the tree.

EXAMPLE:

Postfix Expression: **AB+CD-***

Step	Symbol	Action	Stack (Top → Bottom)
1	A	Operand → Push	A
2	B	Operand → Push	B A
3	+	Operator → Pop B, A Make '+' (B = Right, A = Left)	+
4	C	Operand → Push	C +
5	D	Operand → Push	D C +
6	-	Operator → Pop D, C Make '-' (D = Right, C = Left)	- +
7	*	Operator → Pop (-), (+) Make '*' ((-)=Right, (+)=Left)	*

EXPRESSION TREE:



Legend:

Red → Operator

Green → Operand

QUICK MEMORY TRICK

POSTFIX

Scan: Left to Right

Operator:

First Pop → **Right** Child

Second Pop → **Left** Child



Semester: 2

Subject : Data Structures using Java

Chapter : Tree

❖ Heap

→ A **Heap** is a specialized **Complete Binary Tree** that satisfies the **Heap Property**.

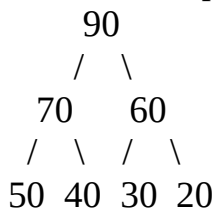
→ A Complete Binary Tree is a tree in which:

- All levels are completely filled except possibly the last level.
- The last level is filled from left to right.

➤ Types of Heap

1. Max Heap

In a Max Heap, the value of every parent node is **greater than or equal to** its children.

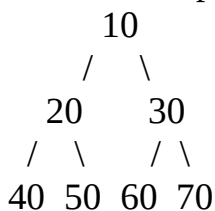


Properties:

- Root contains the largest element.
- Parent $\geq$ Children

2. Min Heap

In a Min Heap, the value of every parent node is **less than or equal to** its children.



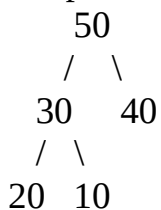
Properties:

- Root contains the smallest element.
- Parent $\leq$ Children

→ Array Representation of Heap

Heap is usually stored using an array.

Example:



Array:

Index : 0 1 2 3 4

Value : 50 30 40 20 10

For node at index i:
Left Child = $2i + 1$
Right Child = $2i + 2$
Parent = $(i - 1)/2$



Semester: 2

Subject : Data Structures using Java

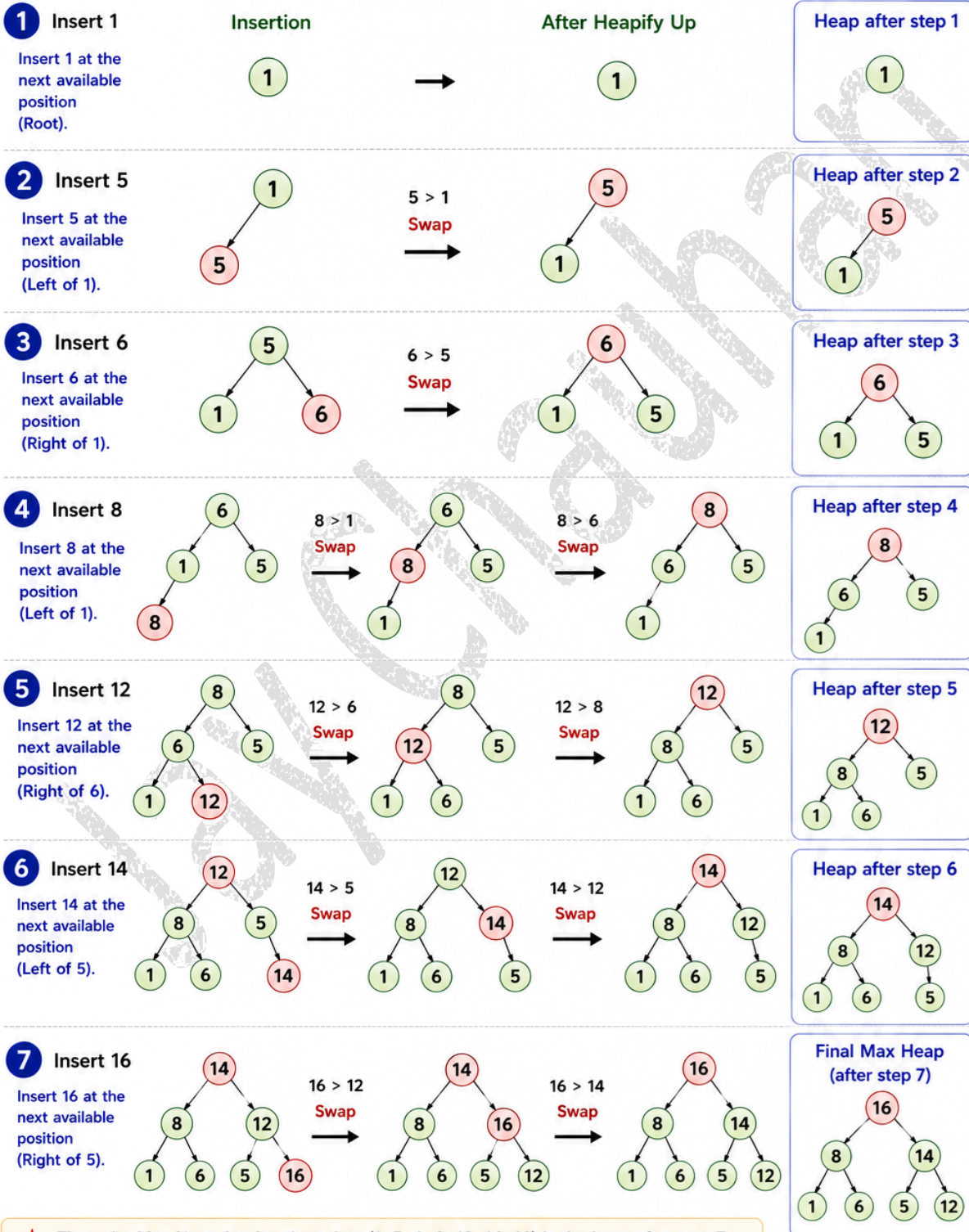
Chapter : Tree

MAX HEAP GENERATION (Successive Insertion)

Create the max heap for the following data:

1, 5, 6, 8, 12, 14, 16

In a Max Heap, parent is always **greater than or equal** to its children.



★ Thus, the Max Heap for the given data (1, 5, 6, 8, 12, 14, 16) is the heap after step 7.



Semester: 2

Subject : Data Structures using Java

Chapter : Tree

MIN HEAP GENERATION (Successive Insertion)

Create the min heap for the following data:

15, 10, 20, 17, 8, 25, 5

In a Min Heap, parent is always **smaller than or equal** to its children.

Step	Insertion	After Heapify Up	Heap after this step
1 Insert 15 (Root)		$\Rightarrow$	Heap after step 1
2 Insert 10 (Left of 15)		$10 < 15$ Swap $\rightarrow$	Heap after step 2
3 Insert 20 (Right of 10)		$20 \geq 10$ No Swap $\rightarrow$	Heap after step 3
4 Insert 17 (Left of 15)		$17 \geq 15$ No Swap $\rightarrow$	Heap after step 4
5 Insert 8 (Right of 15)		$8 < 15$ Swap $\rightarrow$ $8 < 10$ Swap $\rightarrow$	Heap after step 5
6 Insert 25 (Left of 20)		$25 \geq 20$ No Swap $\rightarrow$	Heap after step 6
7 Insert 5 (Right of 20)		$5 < 20$ Swap $\rightarrow$ $5 < 8$ Swap $\rightarrow$	Final Min Heap (after step 7)

★ Thus, the Min Heap for the given data (15, 10, 20, 17, 8, 25, 5) is the heap after step 7.



Semester: 2

Subject : Data Structures using Java

Chapter : Tree

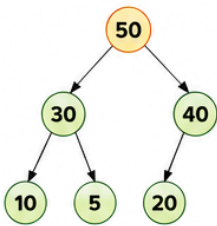
MAX HEAP (Parent $\geq$ Children)

In Max Heap, the value of every parent node is greater than or equal to its children.
The **largest** element is at the root.

MAX HEAP INSERTION

Insert: 55

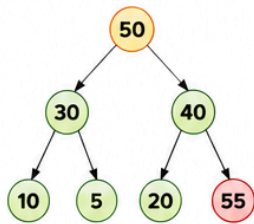
1 Initial Max Heap



Array Representation

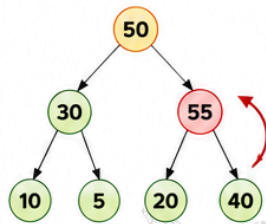
Index : 0 1 2 3 4 5
Value : 50 30 40 10 5 20

2 Insert 55 at last position



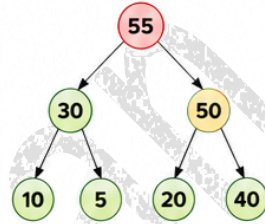
Index : 0 1 2 3 4 5 6
Value : 50 30 40 10 5 20 55

3 55 > 40 Swap



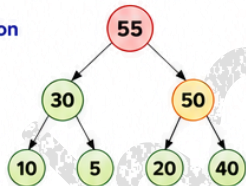
Index : 0 1 2 3 4 5 6
Value : 50 30 55 10 5 20 40

4 55 > 50 Swap



Index : 0 1 2 3 4 5 6
Value : 55 30 50 10 5 20 40

Final Max Heap after Insertion

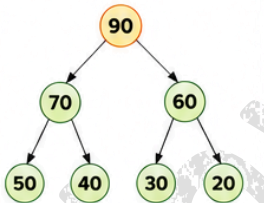


Max Heap Array Representation

Index : 0 1 2 3 4 5 6
Value : 55 30 50 10 5 20 40

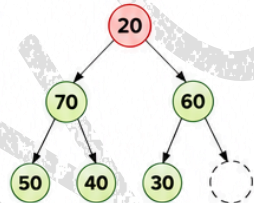
MAX HEAP DELETION (Delete Root)

1 Initial Max Heap



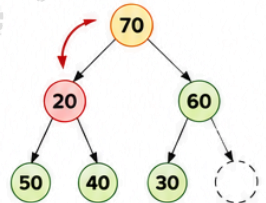
Index : 0 1 2 3 4 5 6
Value : 90 70 60 50 40 30 20

2 Replace root with last node (20)



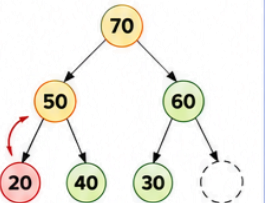
Index : 0 1 2 3 4 5
Value : 20 70 60 50 40 30

3 20 < 70 Swap



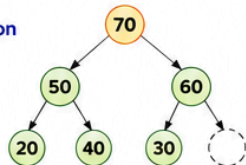
Index : 0 1 2 3 4 5
Value : 70 20 60 50 40 30

4 20 < 50 Swap



Index : 0 1 2 3 4 5
Value : 70 50 60 20 40 30

Final Max Heap after Deletion



Max Heap Array Representation

Index : 0 1 2 3 4 5
Value : 70 50 60 20 40 30

KEY POINTS

- Max Heap $\rightarrow$ Parent $\geq$ Children
- Largest element is always at the root
- It is a Complete Binary Tree
- Insertion $\rightarrow$ Insert at last and Heapify Up (Bottom to Top)
- Deletion $\rightarrow$ Delete root, replace with last node and Heapify Down (Top to Bottom)

ARRAY INDEX FORMULA (0-BASED INDEXING)

- Left Child = $2i + 1$
- Right Child = $2i + 2$
- Parent = $(i - 1) / 2$

Where i is the index of the node

LEGEND

- Existing Node
- New/Moved Node
- Unchanged Node
- Deleted/Empty



Semester: 2

Subject : Data Structures using Java

Chapter : Tree

MIN HEAP (Parent $\leq$ Children)

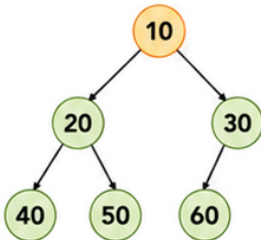
In Min Heap, the value of every parent node is less than or equal to its children.

The **smallest** element is at the root.

MIN HEAP INSERTION

Insert: 15

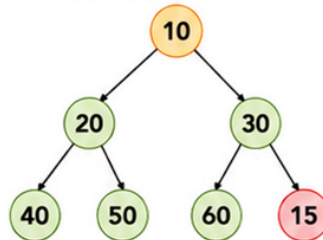
1 Initial Min Heap



Array Representation

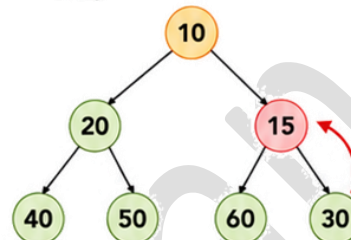
Index	0	1	2	3	4	5
Value	10	20	30	40	50	60

2 Insert 15 at last position



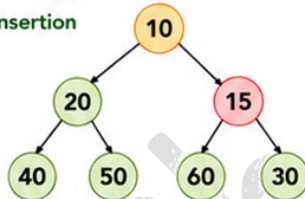
Index	0	1	2	3	4	5	6
Value	10	20	30	40	50	60	15

3 15 < 30 Swap



Index	0	1	2	3	4	5	6
Value	10	20	15	40	50	60	30

Final Min Heap after Insertion

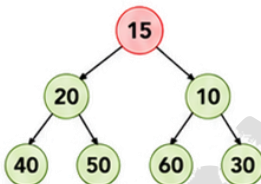


Min Heap Array Representation

Index	0	1	2	3	4	5	6
Value	10	20	15	40	50	60	30

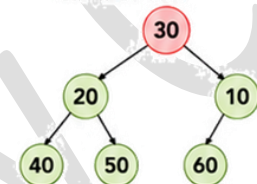
MIN HEAP DELETION (Delete Root)

1 Initial Min Heap



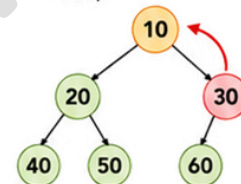
Index	0	1	2	3	4	5	6
Value	15	20	10	40	50	60	30

2 Replace root with last node (30)



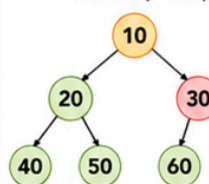
Index	0	1	2	3	4	5	6
Value	30	20	10	40	50	60	30

3 30 > 10 Swap



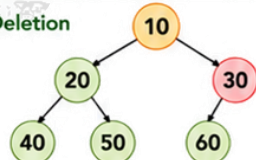
Index	0	1	2	3	4	5	6
Value	10	20	30	40	50	60	30

4 20 > 10 No Swap (Stop)



Index	0	1	2	3	4	5	6
Value	10	20	30	40	50	60	30

Final Min Heap after Deletion



Min Heap Array Representation

Index	0	1	2	3	4	5	6
Value	10	20	30	40	50	60	30

KEY POINTS

- Min Heap $\rightarrow$ Parent $\leq$ Children
- Smallest element is always at the root
- It is a Complete Binary Tree
- Insertion $\rightarrow$ Insert at last and Heapify Up (Bottom to Top)
- Deletion $\rightarrow$ Delete root, replace with last node and Heapify Down (Top to Bottom)

ARRAY INDEX FORMULA (0-BASED INDEXING)

Left Child = $2i + 1$

Right Child = $2i + 2$

Parent = $\lfloor (i - 1) / 2 \rfloor$

i = Where i is the index of the node.

LEGEND

- Existing Node
- New/Moved Node
- Unchanged Node
- Deleted/Empty



Semester: 2

Subject : Data Structures using Java

Chapter : Tree

BINARY TREE TYPES – PROPERTIES, FORMULAS & EXAMPLES

QUANTITY	BINARY TREE / 2-TREE	PERFECT BINARY TREE	FULL / PROPER / STRICT BINARY TREE	COMPLETE BINARY TREE
Rules / Properties	Maximum 2 children allowed	All levels completely filled and all leaf nodes at same level	Every node has either 0 or 2 children	All levels complete except last and filled from left side only
Maximum Nodes	$N_{\max} = 2^{(h+1)} - 1$	$N_{\max} = 2^{(h+1)} - 1$	$N_{\max} = 2^{(h+1)} - 1$	$N_{\max} = 2^{(h+1)} - 1$
Minimum Nodes	$N_{\min} = h + 1$	$N_{\min} = 2^{(h+1)} - 1$	$N_{\min} = 2h + 1$	$N_{\min} = 2^h$
Maximum Height	$h_{\max} = N - 1$	$h_{\max} = \log_2 (N + 1) - 1$ or $h_{\max} = (\log (N + 1) / \log 2) - 1$	$h_{\max} = (N - 1) / 2$	$h_{\max} = \log N / \log 2$ or $h_{\max} = \lceil \log_2 N \rceil$
Minimum Height	$h_{\min} = (\log (N + 1) / \log 2) - 1$	$h_{\min} = (\log (N + 1) / \log 2) - 1$	$h_{\min} = (\log (N + 1) / \log 2) - 1$	$h_{\min} = (\log (N + 1) / \log 2) - 1$
Possible no of Tree	$\frac{(2N)!}{(N+1)!N!}$	-----	-----	-----
Maximum node at level = 2^L	Maximum node at level = 2^L			
Example	Height = 2, Nodes = 6	Height = 2, Nodes = 7 (All levels full, all leaves at same level)	Height = 3, Nodes = 11 (Every node has 0 or 2 children, but leaves not at same level)	Height = 2, Nodes = 5 (Last level filled from left)

SYMBOLS USED	
Symbol	Meaning
N	Total number of nodes
L	Number of leaf nodes
I	Number of internal nodes
h	Height of tree
L (level)	Level number

IMPORTANT FORMULAS (ONLY FOR FULL / PROPER / STRICT BINARY TREE)	
Purpose	Formula
Leaf and internal node relation	$L = I + 1$
Total nodes using leaves	$N = 2L - 1$
Leaf nodes formula	$L = N - I$

DEGREE FORMULA EXAMPLE	
Formula: $N = (n_1 \times 1) + (n_2 \times 2) + 1$	Example Suppose: $n_1 = 5$ $n_2 = 10$ Using formula: $N = (5 \times 1) + (10 \times 2) + 1$ $N = 5 + 20 + 1$ $N = 26$
Meaning of Symbols	
Symbol	Meaning
N	Total number of nodes
n_1	Nodes having degree 1
n_2	Nodes having degree 2

ARRAY REPRESENTATION FORMULAS																	
INDEX STARTS FROM 0	INDEX STARTS FROM 1																
<table> <tr> <th>Condition</th><th>Formula</th></tr> <tr> <td>Left child</td><td>$2i + 1$</td></tr> <tr> <td>Right child</td><td>$2i + 2$</td></tr> <tr> <td>Parent</td><td>$\lfloor (i - 1) / 2 \rfloor$ ($i - 1$ / 2 (integer division))</td></tr> </table> Example Tree (Index from 0) Array: [A, B, C, D, E, F, G]	Condition	Formula	Left child	$2i + 1$	Right child	$2i + 2$	Parent	$\lfloor (i - 1) / 2 \rfloor$ ($i - 1$ / 2 (integer division))	<table> <tr> <th>Condition</th><th>Formula</th></tr> <tr> <td>Left child</td><td>$2i$</td></tr> <tr> <td>Right child</td><td>$2i + 1$</td></tr> <tr> <td>Parent</td><td>$\lfloor i / 2 \rfloor$ (i / 2 (integer division))</td></tr> </table> Example Tree (Index from 1) Array: [-, A, B, C, D, E, F, G] (Index 0 is unused)	Condition	Formula	Left child	$2i$	Right child	$2i + 1$	Parent	$\lfloor i / 2 \rfloor$ (i / 2 (integer division))
Condition	Formula																
Left child	$2i + 1$																
Right child	$2i + 2$																
Parent	$\lfloor (i - 1) / 2 \rfloor$ ($i - 1$ / 2 (integer division))																
Condition	Formula																
Left child	$2i$																
Right child	$2i + 1$																
Parent	$\lfloor i / 2 \rfloor$ (i / 2 (integer division))																

NOTES

- h (height) is counted in terms of edges on the longest path from root to deepest node.
- Level of root node = 0 (commonly used).
- For perfect binary tree with height h : $N = 2^{(h+1)} - 1$ and leaf nodes $L = 2^h$